IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 6 June 2025, accepted 21 June 2025, date of publication 24 June 2025, date of current version 8 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3582868

APPLIED RESEARCH

End-to-End Latency Optimization for Containerized ROS 2 Autonomous Driving Software

TOBIAS BETZ[1], HARUN TEPER[2], (Member, IEEE), DOMINIC EBNER[1], MAXIMILIAN LEITENSTERN[1], SIMON SAGMEISTER[1], (Graduate Student Member, IEEE), MARCEL WEINMANN[1], JIAN-JIA CHEN[2], (Senior Member, IEEE), AND MARKUS LIENKAMP[1], (Member, IEEE)

[1]Institute of Automotive Technology, TUM School of Engineering and Design, Technical University of Munich, 85748 Garching, Germany
[2]Design Automation for Embedded Systems Group, Faculty of Informatics, TU Dortmund University, 44227 Dortmund, Germany

Corresponding author: Tobias Betz (tobias94.betz@tum.de)

ABSTRACT One of the most significant shifts in software development is the growing prevalence of microservice-oriented and containerized applications. However, it faces challenges regarding timing and latency optimization. We present an optimization approach that has been specifically tailored to the use of containerized ROS 2 architectures. The optimization approach was implemented and evaluated using three different configurations, differing by the weighting of individual callback sequences of the system during optimization. Our experimental evaluation demonstrated the optimization of an autonomous racing software, developed for high-speed racing applications. The optimization yields a notable improvement in the maximum end-to-end latency by 22.47 % compared to the baseline measurement.

INDEX TERMS Container, end-to-end latency, robot operating system 2, scheduling, software architecture.

I. INTRODUCTION

The increasing complexity of autonomous driving systems demands highly efficient and reliable software frameworks. Autonomous racing is an ideal test scenario for robust software stacks in edge-case scenarios. Here, the need for low latency is of paramount importance as the vehicles are closely interconnected and interacting. Big challenges in racing scenarios are localization, estimating the dynamic state of the vehicle, and trajectory control in highly dynamic situations. Therefore, one of the most crucial computation chains is the pipeline from localization to control. Delays in data transport and the subsequent calculation of the own position harm control performance, potentially leading to crashes [1]. To tackle these challenges, a suitable software architecture is of vast importance. Hence, a containerized software architecture approach is also gaining prominence in the industry with software-defined vehicles (SDVs) on the rise. In a competitive challenge like racing, containerization offers the benefit of fast development cycles since software dependencies are isolated for each module and can be developed for each module independently. Moreover, it facilitates rapid and efficient software deployment, which is advantageous in scenarios where testing time is limited. Fitting metrics are essential to address timing challenges when deploying complex autonomy systems. The cause-effect chain (derived from event-chains in [2]) is often used to define data propagation latencies in such interconnected systems. It traces the flow of data through the system from sensor inputs to actuator outputs. Metrics such as response time [3], [4], [5] (the duration between sampling data and its corresponding action), reaction time [6] (the duration between an event and the first output based on sampling the event) and data age [7] (the time between sampling data and an output being based on that data) are critical in quantifying system performance. This issue has been previously examined by

The associate editor coordinating the review of this manuscript and approving it for publication was Yangmin Li.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/



FIGURE 1. The software used for analysis and optimization was developed by TUM Autonomous Motorsport for the usage in the A2RL racing competition.

numerous researchers in the academic literature. However, there is a lack of systematic optimization approaches for complex ROS 2 systems.

In Teper et al. [8] a timing analysis and optimization is proposed for distributed ROS 2 multi-executor systems. The optimization is shown on a simple ROS 2 application focusing on a single cause-effect chain. We build upon this approach and present an optimization for containerized ROS 2-based autonomous driving systems. This involves the extension to multiple (prioritized) cause-effect chains, the usage of intra-process communication (IPC), and container usage. The method is demonstrated on an autonomous racing software developed by the TUM Autonomous Motorsport (TAM) team [9], which was successfully used in the Indy Autonomous Challenge (IAC) [10] and Abu Dhabi Autonomous Racing League (A2RL) [11] (shown in Fig. 1) reaching speeds over 250 km h^{-1}. In total, the paper comprises the following main contributions:

- We extend the optimization problem to determine an optimal assignment of processes to containers.
- We integrate IPC communication to consider lower latencies for grouped nodes.
- We evaluate different prioritizations for systems with a large number of chains.
- We present a microservice-based deployment strategy for an autonomous race car.
- We demonstrate and evaluate the framework on a ROS 2 software stack designed for real-world high-speed autonomous racing.

II. RELATED WORK

Advancements in autonomous driving and robotics rely heavily on efficient middleware frameworks like ROS 2 to achieve real-time performance and scalability. However, the complex interaction between software architecture, timing requirements, and containerized environments introduces significant challenges in optimizing latency and ensuring reliable execution.

A. ROS 2 SYSTEMS & APPLICATION

ROS 2 is widely adopted in various domains [12], ranging from humanoid [13] and mobile robotics [14] to unmanned aerial systems [15]. While facing a lot of challenges in developing autonomous driving architecture [16], many research institutions and industrial organizations leverage ROS 2 for their projects. Integrating it with industry standards such as AUTOSAR to meet safety-critical requirements makes it a suitable solution for developing autonomous vehicles [17]. Autoware [18], [19], a prominent open-source project, shows ROS 2's capabilities in enabling advanced autonomous driving systems, offering software packages for perception, planning, and control. To achieve optimal performance, researchers have explored architectures that incorporate hardware acceleration for ROS 2 applications [20]. These solutions use FPGAs, GPUs, and specialized processing units to accelerate computationally intensive tasks such as perception, simultaneous localization and mapping (SLAM), and deep learning inference. An emerging area of research is autonomous racing, which pushes the limits of autonomous vehicles in terms of speed, robustness, and performance. Competitions like IAC [10] and A2RL [11] tackle these challenges. Their high-speed environments require fast decision-making and reliable control systems, resulting in a need for a robust framework like ROS 2. Numerous publications have demonstrated its application in competitive scenarios where teams optimize algorithms for path planning, vehicle dynamics, and obstacle avoidance [9], [21]. A critical aspect of the ROS 2 ecosystem is the availability of simulation tools to facilitate the development and validation of robotic applications. Simulators such as Gazebo [22], CARLA [23], LGSVL [24], and AWSIM [25] provide comprehensive environments for testing ROS 2 systems. These tools often include bridges [26] to synchronize with autonomous driving software stacks.

B. ROS 2 LATENCY EVALUATION

To gain a better system performance, latency analysis has to be performed to understand possible bottlenecks. Compared to ROS 1, ROS 2 offers better real-time performance through Data Distribution Service (DDS) [27]. In [28] ROS 1 and ROS 2 are evaluated in terms of their communication latency. Middleware optimizations were identified as a driving factor in improving real-time performance in autonomous driving. Various ROS 2 tools are proposed in order to measure latency. Bedard et al. [29] present a ROS 2 tracing framework with debugging and performance analysis tools which help to analyze systems [30]. In [31], a tool for analyzing message flows and identifying bottlenecks is presented, focusing on nodes, inter-node communication, callback, and end-to-end latencies. Similarly, [32] presents an enhanced framework based on [29] for latency analysis. Manual dependency definitions are obsolete with the approach described in [33], which allows precise measurement of end-to-end and component-level latencies. Research also has focused on

investigating autonomous software stacks and their latency on different hardware platforms with the goal of minimizing latency in embedded systems. Reference [18] introduces a modified profile that balances computational loads, resulting in acceptable latency for real-world applications. DDS middleware implementations have been analyzed in [34]. Reference [35] presents a novel IPC technique that reduces communication latency. ROS 2-based software stacks were analyzed to identify bottlenecks in various software modules in terms of their impact on latency [1]. Reference [36] analyzed system parameters across different configurations. Reference [37] investigates latency, message loss, and buffer size issues in ROS 2 systems. Case studies show that a callback prioritization heuristic and modifications to the ROS 2 executor reduce end-to-end latency, improve memory efficiency, and increase system reliability. Furthermore, in resource-constrained and high-frequency communication scenarios, node composition significantly reduces resource consumption and latency [38].

C. ROS 2 TIMING ANALYSIS

Real-time systems and robotics are advancing rapidly, which drives the need for predictable and efficient timing behavior for complex software stacks as needed for SDVs. Research provides several contributions to this topic. One of them is RT-ROS, a hybrid architecture enabling the execution of real-time and non-real-time ROS tasks concurrently on multi-core processors using separate cores. RT-ROS achieves significant reductions in latency as demonstrated in [39]. Furthermore, frameworks like ROSCH address critical latency requirements for autonomous driving systems. It ensures end-to-end latency and timestamp synchronization while maintaining compatibility with legacy ROS applications [40]. Similarly, for ROS 2, PiCAS tries to minimize end-to-end latency by prioritizing callback chains. PiCAS is able to demonstrate significant improvements in latency and schedulability which makes it suitable for time-critical applications. The analysis of response times, especially for ROS 2 systems has received considerable attention in research. Reservation-based scheduling models provide a framework for designing predictable and efficient real-time applications [3]. In [4] an advanced analytical model is presented that addresses variability in execution times and exploits starvation freedom in ROS 2's callback scheduling. Tang et al. [41] show that optimizing the priority of sink callbacks helps to reduce worst-case and average response times for real-time systems. To enhance ROS 2s capabilities dynamic scheduling strategies are applied. In [42] a deadline-based dynamic scheduling scheme for the ROS 2 executor is proposed. Probabilistic approaches guarantee upper bounds on end-to-end latencies, even for high CPU utilization [43] and in [44] a novel scheduling synthesis for multi-core processors is introduced to optimize probabilistic end-to-end latencies in AD software stacks. It uses task phase optimization to improve response times. Günzel et al. [45] extend end-to-end latency analyses to incorporate probabilistic guarantees which provide practical bounds that are validated through models and benchmarking. In [46] analytical models are introduced to improve schedulability and reduce response times for multi-threaded executors. The risk of starvation in multi-threaded executors has also been addressed with proposing a minimal redesign of the executor to ensure starvation freedom validated through empirical testing and formal proofs [47]. Fundamental work on end-to-end latency in [7] shows that different timing metrics are equivalent, simplifying the analysis and measurement process. Inter-process communications delays in ROS 2 have been modeled to provide a deeper understanding of factors influencing ROS 2's real-time performance [48]. In [49] a model is designed to bound DDS delivery latency in distributed real-time systems. Reference [6] focuses on analyzing the timing guarantees of cause-effect chains in ROS 2 systems. Another work proposes system optimizations to address latency and memory inefficiencies by bounding timer periods and subscription buffer sizes, ensuring better performance through callback prioritization [37]. Finally, [8] proposes a timing analysis for multi-executor ROS 2 systems and optimizes end-to-end latencies. It achieves significant latency reductions in autonomous racing software stacks for a single cause-effect chain.

D. VIRTUALIZED SYSTEMS & CONTAINERIZATION

The automotive industry is undergoing a significant transition towards the adoption of SDVs, a shift that is being driven by initiatives such as the Scalable Open Architecture for the Embedded Edge (SOAFEE) [50]. The dynamic updating and reconfiguration of SDVs demands the incorporation of two fundamental technologies: containerization and virtualization. The impact of performance on these systems has been extensively researched in the domains of IoT and cloud computing, as evidenced by [51]. This study introduced a semi-automated benchmarking suite to assess the performance implications of virtualization technologies. As SDVs become increasingly pertinent to the automotive sector, enhancing performance, safety, and security becomes paramount, as discussed in [52]. Containerization has been found to be a suitable approach for high-performance computing in existing research, as referenced in [53]. One of the most common container toolbox is Docker. A comprehensive overview of capabilities for running applications within portable, lightweight containers can be found in [54]. The aforementioned study highlights Docker's ability to ensure consistent environments and observes that containers generally offer a minimum equal or even better performance than virtual machines for CPU and memory tasks. In the context of the automotive sector, Lotz et al. [55] underscore the merits of microservice architectures for ADAS systems, citing their capacity to address challenges such as escalating complexity and implementation costs. Additionally, they highlight the ability to foster modularity, scalability, and independent development. Insights on enhancing microservice-based

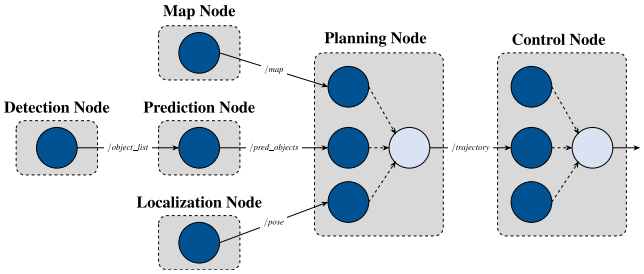


FIGURE 2. Example of a ROS 2 architecture. Dotted lines represent intra-node communication via shared variables. The light-blue circles represent timer callbacks. Adapted from [33].

applications are offered in [56]. Containerization and virtualization have been demonstrated as viable solutions for SDVs, as evidenced by the findings of Wen et al. [57]. In a study, Betz et al. [58] implemented a microservice architecture for a full autonomous driving stack based on ROS 2. The merits of containerization for SDVs have also been demonstrated in the works of [59] and [60]. Reference [61] uses the microservice architecture presented in [58] and extends it by an automated approach for resource allocation and software deployment. They use model-based system engineering and solve an optimization problem. In addition to the aforementioned advantages, research findings indicate that virtualization and containerization result in minimal performance degradation with respect to CPU, memory, and network usage, as outlined in the study by Wen et al. [62]. To provide a comprehensive perspective on container scheduling strategies, we incorporate the survey by Ahmad et al. [63], which offers a systematic classification of scheduling techniques, including mathematical, heuristic, meta-heuristic, and machine learning approaches, and highlights their applicability to real-time and resource-constrained environments.

III. ROS 2 ARCHITECTURE

This chapter provides an overview of the foundational ROS 2 architecture, explaining the core concepts that enable its versatility.

A. MAIN CONCEPTS AND COMPONENTS

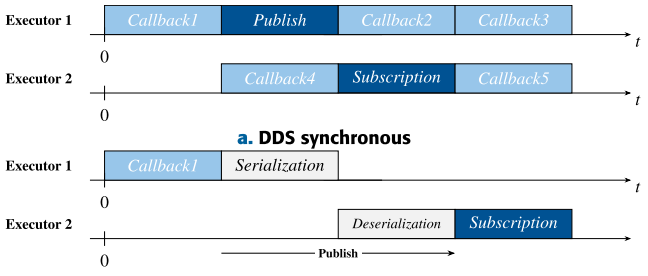
In ROS 2, the fundamental building block of a system is the *node*, representing a system component, such as motion planning or sensor data processing. Each node consists of a set of tasks that define the node's behavior. We consider systems including *timer* and *subscription* tasks, which are the most common types of tasks used in ROS 2 systems. Timers are periodically activated time-triggered tasks, while subscriptions are event-triggered tasks that are activated whenever a message is received. Each task has a corresponding *callback*, which is a function that is executed when the task is scheduled. The communication in ROS 2 is based on the publish-subscribe model of the DDS standard. ROS 2 uses one-way broadcast communication, where callbacks can send messages to so-called topics via so-called publishers, which are then broadcasted to all subscriptions listening to the topic. Nodes can have multiple publishers and subscribers, and tasks can publish messages to multiple topics through their callbacks using publishers. In ROS 2, subscriptions subscribe to and receive messages from only one topic. An example of a ROS 2 architecture is shown in Fig. 2, which illustrates a system with multiple nodes communicating through topics.

B. ROS 2 EXECUTOR SCHEDULING

The scheduling of tasks in ROS 2 is managed by *executors*. Each node and its tasks are assigned to exactly one executor, which is responsible for determining the order in which activated tasks are executed. Furthermore, each task has a priority that the executor uses to determine the order of execution. We focus on the single-threaded executor, which processes tasks sequentially in a single thread. While other executors are available, the single-threaded executor is most widely used and has more analysis and optimization strategies available in the literature. An executor operates an event loop that iteratively awaits, samples, and processes events, such as the arrival of messages or the expiration of timers. Specifically, it manages a *wait set* to store the instances of the tasks that are pending execution and processes these tasks in a sequential manner. The executor involves a two-phase design, consisting of *polling points* and *processing windows*. During each polling point, the executor samples one instance of each task that is activated, either through an expired timer period or the presence of a message in a subscription buffer. After the polling point, a processing window is initiated during which the executor processes all sampled task instances in the wait set. ROS 2 natively prioritizes timer callbacks over subscription callbacks. Callbacks of the same type are executed in the order in which the tasks are added to the node and executor. The prioritization is a key aspect in the optimization of the ROS 2 system to reduce end-to-end latency.

C. ROS 2 COMMUNICATION

We will now introduce the communication mechanisms in ROS 2 that are essential for the interaction between nodes. The most common form of communication in ROS 2 is the publish-subscribe model, where nodes communicate through topics. Here, publishers send messages to topics that are then broadcast to all subscribers listening to the topic. This form of communication can be used for nodes running in separate processes as well as for nodes running within the same process. In most cases, nodes run as separate processes, each with its own memory space. Therefore, nodes cannot directly access each other's memory. This form of communication is managed through DDS over the network, where data must be *serialized* by the publisher, transmitted, and *deserialized* by the subscriber. In the following, we refer to this kind of communication as DDS communication. When nodes run within the same process, they can communicate directly through shared memory, bypassing the DDS middleware. This form of communication is called *intra-process*



b. DDS serialization and deserialization of the message

FIGURE 3. Schematic of the ROS 2 communication behavior.

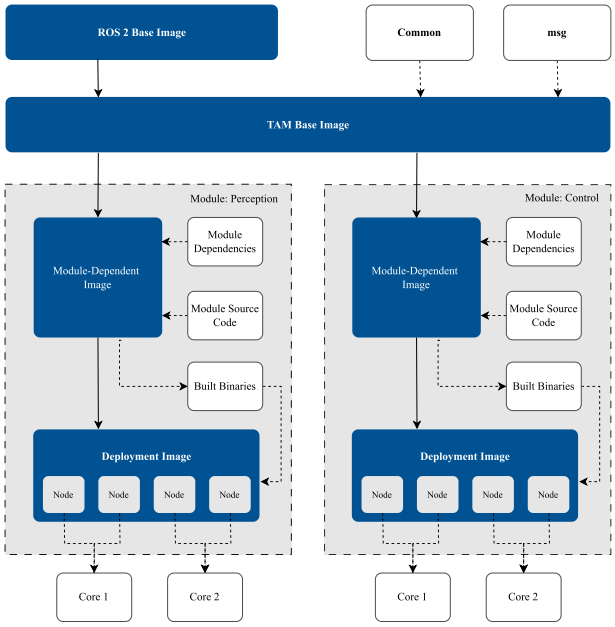


FIGURE 4. Build process for two example services in the used TAM microservice architecture. For deployment the ROS 2 node components are composed on runtime and pinned to the individual cores.

communication (IPC) and is more efficient than DDS communication, as it avoids the overhead of serialization and deserialization. In this paper, the assignment of nodes to processes running in separate containers will be a key aspect of the optimization strategy to reduce end-to-end latency. For this, the optimization will need to consider the trade-off between the scheduling overhead when combining nodes in the same container and the communication latency when separating them across containers. While ROS 2 provides many quality of service options for DDS communication, we focus on synchronous communication, where the sending thread is blocked until the data has been transmitted over the network, thus including the communication latency as part of the execution time of the task. Other options, such as asynchronous communication, use separate threads for DDS communication. However, these are harder to analyze.

IV. MICROSERVICE CONTAINER ARCHITECTURE

In the domain of ROS 2 systems, there is an observed trend towards microservice-based architectures due to their enhanced scalability, modularity, and accelerated development options. Microservice-based architectures adopt a containerization approach, wherein distinct software functionalities are divided into separate containers. Each container is defined with its own dependencies, ensuring that conflicts between modules are avoided. Consequently, these architectures offer a reproducible and isolated environment for each service. The benefits of this approach include the isolation of dependencies and the simplified management of versions, further improved through the use of `rosdep`, an automatic ROS 2 package dependency installation tool. Additionally, the development speed can be enhanced due to the ability to handle dependencies between software components in an obsolete manner. Modules can be easily modified based on different versions, provided the interfaces between containers remain unchanged. This feature is particularly advantageous for the development of software stacks. From the perspective of autonomous racing, the accelerated development timeline and versioning of software are particularly advantageous in facilitating efficient testing of specific software iterations. To deploy multiple containers on target hardware, various orchestration tools such as `kubernetes` [64], `k3s` [65], and `docker compose` [54] are available.

Fig. 4 illustrates the build process for two exemplary services used in the TAM's microservice architecture. The process starts with the publicly available ROS 2 image. To this image, we add the required message definitions and our common library, which is required for all downstream-built microservices. These dependencies are built into a new image which we refer to as the TAM base image. In addition, we build a module-specific image by including the dependencies for a specific microservice. This image still does not contain any source code for the microservice itself. Rather, we use the resulting image to build a module's source code locally utilizing a bind mount. After building locally, we use the resulting binaries and the module-specific base image to create the so-called deployment image. This contains only the microservice's binaries and the required dependencies. By following this scheme, it is possible to drastically reduce the download size of containers. This is because the TAM base image, as well as a module's dependencies, remain unchanged for long periods. The layered approach of docker containers enables to only download the last layer containing the microservice-related binaries, e.g., while executing an over-the-air update.

After building all required microservice containers, it is necessary to distribute and schedule them on the compute unit; a point at which the paper becomes relevant. We propose a methodology for an optimized allocation of these components to containers, intending to minimize latency. To allow for time-efficient testing of the optimized compute layouts, we build all nodes as `rclcpp_components`. This allows runtime composition of the individual nodes, tremendously simplifying the deployment in a specific configuration.

V. END-TO-END LATENCY IN ROS 2

Given the architecture of ROS 2 and its components, we now describe the metric that we use to determine the latencies in ROS 2 systems. For this, we introduce the end-to-end latency in ROS 2 systems as described by Teper et al. [8]. In order to formulate the end-to-end latency the cause-effect chain is introduced. It is a sequence of tasks that propagate data through the system. The cause-effect chain E_i is defined as $E_i = \{\tau_1, \tau_2, \ldots, \tau_{|E_i|}\}$, where τ_1 is the first task in the chain and $\tau_{|E_i|}$ is the last task in the chain. Each task in the chain directly communicates with the next task in the chain. The end-to-end latency of the cause-effect chain E_i is defined as $L_i = \sum_{j=1}^{|E_i|} L(\tau_j)$, where $L(\tau_j)$ is the end-to-end latency of task τ_j. For the single-threaded executor, [8] provide a method to compute the upper bound on the end-to-end latency of task τ_j, consisting of two individual upper bounds. First, ub_j^{pre} is the upper bound of the time between the time the data is available, i.e. the message is received, and when the task starts being executed. Second, ub_j^{exe}, is the upper bound of the time the task takes to execute after the start of the execution. This includes the time to send the message to the next task in the chain, if necessary. For each task type, different upper bounds are provided in Lemmas VI.1-VI.8 in [8]. In the following, we provide the parameters that influence the end-to-end latency of a task in ROS 2.

A. SCHEDULING LATENCY

For the scheduling latency ub_j^{pre}, there are three aspects that influence the end-to-end latency:

- Timer periods: These influence how long timer tasks may idle before being activated. They can be optimized to control the execution frequency. However, while the execution frequency might be improved, the maximum delays caused by other tasks are not affected by the timer period.
- Task Prioritization: The executor can be configured to either prioritize timers over subscriptions or subscriptions over timers. Furthermore, the order in which nodes and callbacks are registered in the executor affects the task prioritization as well, as elaborated in [8]. This affects the delays caused by interfering tasks.
- Task interference: The assignment of tasks to the same executor causes interference between those tasks. In the ideal case, all nodes are assigned to an individual executor, minimizing interference. However, as each executor represents a separate process, the scheduling latencies of the containerization need to be considered as well. Therefore, we include it in the optimization problem and it is discussed in VI-B2.

B. COMMUNICATION LATENCY

For the upper bound ub_j^{exe}, the execution time of the task τ_j consists of the time it takes to execute the functionality of the task and the time it takes to send messages. We do not consider the optimization of the execution time of the task τ_j in this work, as we assume that the execution time is fixed and not influenced by the system configuration. However, we consider the optimization of the communication latency, which is the time it takes to send messages between tasks. As shown by Macenski et al. [38], IPC can reduce the communication latency between nodes greatly when compared to DDS communication. Specifically, we consider the communication latency between nodes within the same process and between nodes in different processes. For this, the optimization determines the assignment of nodes to executors and the assignment of executors to processes to minimize the communication latency between nodes. If nodes are assigned to the same process, IPC communication is used, minimizing the latencies. If nodes are assigned to different processes, the communication latency is increased, as the communication between nodes is done through DDS communication. In the most extreme case, the optimization would group all nodes into the same executor to minimize the communication latency. However, as nodes are grouped into one executor, the scheduling latencies of ub_j^{pre} increase, as more tasks may interfere with each other. For the optimal solution, a balanced assignment of nodes to executors is necessary to minimize both the communication latency and the scheduling latencies.

VI. OPTIMIZATION

In this section, we present the optimization. First, we describe the system model and chain generation. Subsequently, the optimization parameters are discussed followed by an overview of the general optimization workflow.

A. SYSTEM MODEL AND CHAIN GENERATION

For the system model, the following static configuration is given by the system code and implementation. Therefore, the following is required for optimization:

- The unique nodes and their callbacks, including the callback-to-node assignment.
- The period of each timer.
- The subscription topic of each subscription.
- The message size of each publisher.
- The programming language of each node. A mixture of Python and C++ nodes is not possible. Also, Python nodes can be assigned to the same executor, but cannot use IPC as it is only available in C++ through `rclcpp`.

Furthermore, we need to specify the constraints that limit nodes to individual executors or processor cores. In the context of sensor drivers, it is imperative to ensure their placement is executed in an isolated manner. Failure to do so can result in a disruption to the system's overall performance, particularly in instances where the input frequency remains unstable. Another case is when using Python containers. Nodes cannot be put into the same container environment to compose it, as dependencies can be different.

Moreover, we require the following data, gathered by measuring the latencies during execution:

- For each callback, associated with either a timer tmr_i or a subscription sub_i, we require the worst-case execution time and its publishers.
- For each publisher, we require the worst-case communication latency using DDS inter-process communication.

We note that the variance of these latencies are independent of the configuration of the system and can be captured using the available latency measurements tools of ROS 2, such as `ros2_tracing` [29]. For the optimization, we need to determine all chains in the system $E = \{tmr_i, sub_i + 1 \cdots, sub_n + 1\}$ that start with a timer and are only followed by subscriptions thereafter. We assume that every subscription only receives messages from one unique publisher. Starting from each timer, the chains extend by following the DDS publishers and subscriptions. Once a callback of a subscription does not include a publisher that propagates data, the chain is completed. As a result, we receive a set of chains for the optimization.

B. OPTIMIZATION PARAMETERS

In the following, we explain the parameters that are used for optimization. We begin with the weighting factors, then discuss containerization. Subsequently, timer settings and IPC communication is discussed.

1) WEIGHTING OF CALLBACK SEQUENCES

For optimization we minimize the weighted sum of the end-to-end latencies W_i of all chains in the system. For each chain, an end-to-end latency can be determined using the method described in Section VI-A. We denote L_i as the latency of chain i, with N as the total number of chains in the underlying system. Furthermore, we assign an integer value to each chain as its priority. We denote $P_r(i)$ as the priority assigned to chain i. This priority is arbitrarily chosen by the developer and signifies its importance during optimization. Then, we generate a weighted bound and finally sum all weighted bounds up to generate the final optimization value. For the weights, we chose three different weighting schemes: *equal*, *linear*, and *logarithmic*.

For the *equal weighting* all latencies have the same weight. Therefore, the equal weighted latency for each chain is $W_i = L_i$. For the *linear weighting*, each latency is divided by its priority. Therefore the weighted latency for each chain is $W_i = \frac{L_i}{P_r(i)}$. For the *logarithmic weighting*, each latency is divided by a factor of $10^{3 \cdot P_r(i)}$, which leads to a weighted latency for each chain $W_i = \frac{L_i}{10^{3 \cdot P_r(i)}}$. Additionally, weightings according to the longest path (number of nodes) or critical path (highest latency) could be added. In our evaluation, we will observe the impact of each function and determine which one is best suited to optimize our system.

2) CONTAINER OPTIMIZATION

So far, container optimization has not yet been considered in prior optimization approaches, such as [8]. Therefore, we combine the results of previous work on container scheduling and the optimization of the end-to-end latency of the cause-effect chain in ROS 2. Modeling container scheduling behavior is complex, therefore we make several assumptions: Each container is abstracted as a server, and each process running on the server is running a single-threaded executor that processes its assigned tasks. Each server is specified by a period P and a budget B. For our optimization, we assume that resources are equally distributed among the processes running on the server. Therefore, given a server period P and the number of processes n, the server budget is given by $B = \frac{P}{n}$. This assumption is introduced due to the difficulty in determining the exact resource consumption of each process. For a given task latency $L(\tau_j)$, the total time it takes to process the task on the server is given by the service time $S(\tau_j)$. We determine the service time $S(\tau_j)$ by using the supply bound function shown in Theorem 2 by Ueter et al. [66]. For the end-to-end latency of a chain, we redefine the sum of the chain to use the service time instead of the end-to-end latency itself, resulting in $L_i = \sum_{j=1}^{|E_i|} S(\tau_j)$. This way, we extend the end-to-end latency upper bound by considering the container scheduling and the service time of the tasks, and preventing too many executors from being assigned to the same container.

3) TIMER SETTING

For the optimization, we allow two settings for the timer period. Either, the timer period is static and given by the system developer, or the developer provides a discrete number of values the timer period can be set to by the optimization. The developer can then choose whether to use static periods or discrete periods for the optimization.

4) IPC COMMUNICATION

For the optimization of DDS latencies, we need to determine the latencies when either using inter-process or intra-process communication. However, determining the worst-case communication latency of each publisher using IPC is tedious. Specifically, it would require to combine and compile all combinations of nodes that could use IPC, so that they could be run by the same process. This phenomenon is particularly pronounced when utilizing containers, as it necessitates a substantial amount of rebuilding and restructuring of repositories and packages. Therefore, we simplify this problem by approximating the worst-case communication latency using the message size. We determine a function that has prerecorded values of the worst-case communication latency using IPC for specific message sizes and extrapolate the worst-case execution latencies of our publishers from these points. This simplifies the optimization and preparation process, as we do not have to test every node combination to determine our IPC optimization values. In our evaluation, we will determine how this method holds after applying it to our optimization.

C. OPTIMIZATION WORKFLOW

The overall workflow of our optimization is depicted in Figure 5 and consists of the following steps: The system

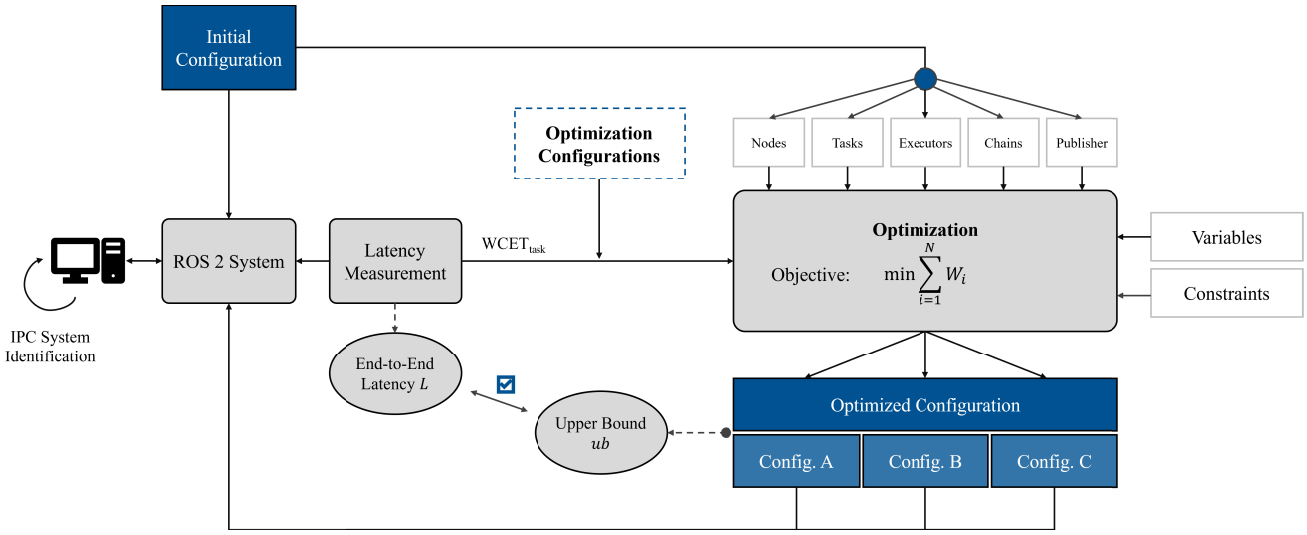


FIGURE 5. A schematic drawing of the proposed optimization workflow. Starting from a default configuration for a specific ROS 2 system, latency measurements are conducted. With an optimization configuration determined by user, a constraint optimization problem is solved leading to an improved ROS 2 system.

configuration is read from the provided code of the nodes. The chains of the system are determined based on the system configuration. An initial measurement of the system is performed to obtain execution and communication latencies from callbacks and publishers, respectively. To ensure the measurement of DDS time exclusively, it is necessary to disable IPC. The end-to-end latencies of the chains of the initial configuration are measured as a reference point. The developer then adds constraints that specify the number of cores of the system, as well as nodes that cannot share a core or executor with other nodes. The system configuration, latency measurements, and constraints are then input for the optimization problem to formulate the constraints of the system. The variables and constraints of the optimization algorithm can be derived from Appendix A. The objective function is specified as a weighted sum of the end-to-end latencies of all chains. The output of this process is an optimized solution that includes a new system configuration. Subsequent to the formulation of the new system configuration, new chain latencies are measured, and the new latencies are compared to the baseline measurement.

VII. EVALUATION

In this section, we describe several experiments using our optimization and present the results.

A. EXPERIMENTAL SETUP

The experiments were conducted on a high-performance computing (HPC) unit for vehicle prototyping. The HPC is equipped with AMD Epyc 7313P (16 x 3.0 GHz) CPU, 128 GB RAM, and NVIDIA RTX A6000 48 GB GPU. A standard Ubuntu 22.04 OS was used without the real-time extension Preempt_RT. Hyperthreading capabilities were disabled, and the CPU clock frequency was set at a constant speed. The software stack is based on ROS 2 *Humble Hawksbill* with Eclipse Cyclone DDS [73]. Given that our use case employs Eclipse Cyclone DDS, we restricted our experiments to synchronous DDS, as it does not provide an option for asynchronous communication [8]. The autonomous driving software stack is in-house developed and

TABLE 1. Description of the individual software services.

Service	Description
LiDAR Pointcloud Filter (LF)	Different stages of pointcloud filtering, including ground filter, voxelization, and geometric filtering.
LiDAR Clustering (LCL)	Object detection pipeline using greedy clustering.
LiDAR Neural Network (LNN)	Object detection pipeline using PointRCNN neural network [67].
LiDAR Localization (LiLo)	KISS-ICP [68] based LiDAR scan matching with the help of an offline generated point cloud map.
RADAR Pointcloud Filter (RF)	Several stages of removing unused point cloud data (e.g., thresh holding, geometric).
RADAR Clustering (RCL)	Object detection pipeline using k-d tree clustering.
RADAR Localization (RaLo)	KISS-ICP based RADAR scan matching with the help of an offline generated point cloud map.
Object Tracking (OT)	Kalman Filter-based object tracking [69] and behavior forecasting of other vehicles.
Sampling Planner (SP)	Generation of a set of trajectories using Frenet samples [70].
Behavior Planner (BP)	Global path planning for general vehicle behavior.
Tracking Controller (TC)	Model-predictive controller [71] for trajectory following and generating high-level control commands.
Steering Controller (SC)	Low-level PID controller for steering commands.
Longitudinal Controller (LC)	Low-level PID controller for acceleration and braking command.
State Estimation (SE)	Kalman-Filter for fusing several sensor information to calculate the vehicle state [72].
Vehicle Gate (VG)	Switch between autonomous mode and overwriting capabilities (e.g., joystick).
Vehicle Interface (VI)	Converting ROS 2 to CAN message.

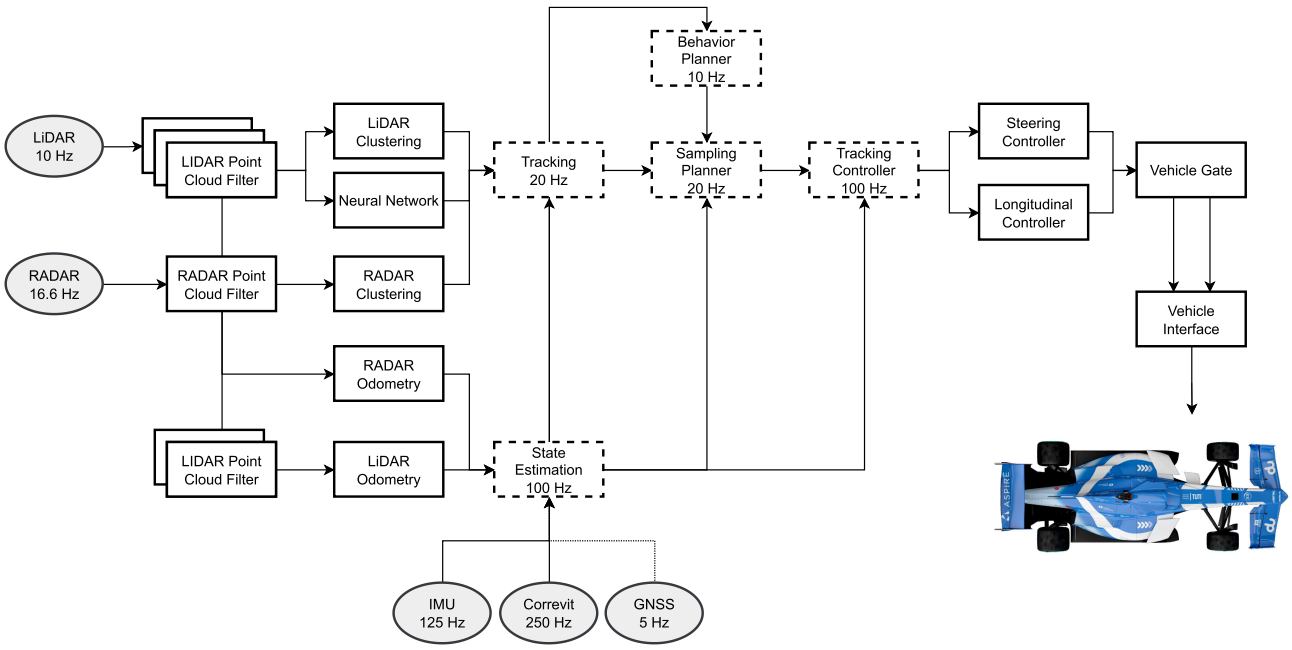


FIGURE 6. Schematic of the autonomous driving software stack used. The dotted boxes represent a node with a timer period. The individual nodes are deployed within individual containers.

designed for autonomous racing competitions. The efficacy of the software configuration has been demonstrated in real vehicles during high-speed interactions. The schematic of the software architecture is shown in Fig. 6. The software is divided into individual containers built in a CI pipeline. We used Docker (version 26.0.0) as containerization technology and `docker-compose` (version 2.25.0) as the orchestration framework. Containers communicate on the local host network. Each node was deployed as a single container and the controller container deployed multiple nodes with a launch file. An overview of the services is given in Table 1. We executed the software stack in a closed loop Unity simulation environment. The simulation ran on a separate simulation machine. LiDAR and RADAR sensor models generated realistic point cloud data. The scan patterns from the LiDAR coincide with the original sensor hardware. RADAR sensors are more complex to simulate; therefore, we used a reduced LiDAR model with more noise. We collected measurement data using `ros2_tracing` [29] and calculated the end-to-end latency with the framework proposed in [33]. The measurement framework is open source and can be accessed online [74]. The end-to-end latency can be decomposed into sub-components according to the definition in Section V to parameterize the timing model and the corresponding optimization.

We chose a scenario where we followed and interacted with another race car. We configured the simulation in such a way that we simulated a GNSS-denied area. Therefore, we need to rely on the LiDAR and RADAR odometry as localization solution. The speed of the leading vehicle was $50\,\mathrm{m\,s^{-1}}$, demonstrating high-speed interaction as shown in Fig. 7. To avoid the impact of changing input frequency of the sensor models on the end-to-end latency, end-to-end latencies have been filtered. We filtered the data in the case of two subsequent time stamps violating the ideal sensor frequency. This led to a data reduction of 1.5 % in the case of LiDAR and 0.6 % for RADAR respectively. In total, each callback sequence for each configuration consisted of approximately 10 000 end-to-end paths.



FIGURE 7. Closed-loop Unity simulation showing the interaction scenario between two vehicles at the Yas Marina Circuit. The orange dots show the LiDAR point cloud used for the perception.

B. BASELINE CONFIGURATION

The baseline measurement, which was utilized for the optimization framework, encompassed the consideration of several configurations, as illustrated in Table 2. The total number of executors distributed across 24 containers was 26, with the baseline measurement being measured on 14 cores. A detailed examination of the configurations reveals that

TABLE 2. Overview about the baseline measurement and parametrization including the callback execution times in ms. Callback types are subscriber (S) or timer (T). Note that LNN, OT, SP, and BP are Python nodes.

Callback	Period	Executor	Container	CPU	$\overline{\text{WCET}}$	$\text{WCET}^{\max}$	Callback	Period	Executor	Container	CPU	$\overline{\text{WCET}}$	$\text{WCET}^{\max}$
LiDAR	100	1	1	1	8.054	24.291	SE:KF/T	10	18	18	3	0.136	0.644
RADAR	60	2	2	2	5.022	10.750	OT:LCL/S	-	19	19	12	0.114	3.047
IMU	8	3	3	3	0.116	1.799	OT:LNN/S	-	19	19	12	0.139	2.611
Correvit	4	4	4	3	0.004	0.260	OT:RCL/S	-	19	19	12	0.002	0.065
LF1:PC/S	-	5	5	4	10.376	16.472	OT:SE/S	-	19	19	12	0.043	0.134
LF2:FPC/S	-	6	6	4	9.221	15.803	OT:KF/T	50	19	19	12	7.295	16.806
LF3:FPC/S	-	7	7	4	3.192	7.750	SP:OT/S	-	20	20	13	0.198	0.404
LF4:FPC/S	-	8	8	4	5.650	10.973	SP:SE/S	-	20	20	13	0.073	0.156
LF5:FPC/S	-	9	9	4	0.053	0.154	SP:BP/S	-	20	20	13	0.179	0.261
LF6:PC/S	-	10	10	5	1.150	3.309	SP:TP/T	50	20	20	13	23.981	48.939
LF7:NPC/S	-	11	11	5	1.991	3.879	BP:SE/S	-	21	21	12	0.039	2.062
LCL:FPC/S	-	12	12	6	14.130	62.361	BP:GP/T	100	21	21	12	5.565	9.679
LNN:FPC/S	-	13	13	6	21.385	25.997	TC:TP/S	-	22	22	14	0.003	0.066
LiLo:NPC/S	-	14	14	7,8,9	47.745	186.494	TC:SE/S	-	22	23	14	0.009	0.033
RF1:PC/S	-	15	15	10	0.701	1.742	TC:CO/T	10	22	22	14	5.520	6.735
RCL:FPC/S	-	16	16	10	0.257	24.856	SC:CO/S	-	23	22	14	0.004	0.017
RaLo:NPC/S	-	17	17	11	59.921	196.558	LC:CO/S	-	24	22	14	0.111	0.219
SE:RaLo/S	-	18	18	3	0.002	0.065	VG:SC/S	-	25	23	14	0.003	0.010
SE:LiLo/S	-	18	18	3	0.002	0.025	VG:LC/S	-	25	23	14	0.004	0.013
SE:IMU/S	-	18	18	3	0.004	0.013	VI:SC/S	-	26	24	14	0.014	0.027
SE:CO/S	-	18	18	3	0.002	0.008	VI:LC/S	-	26	24	14	0.015	0.028

TABLE 3. Overview about the size of the ROS 2 messages with the corresponding publisher and subscriber.

Message	Publisher	Subscriber	$\overline{\text{KB}}$	$\text{KB}^{\max}$
/sensor/lidar	LiDAR	LF1:PC/S	5530	6240
/perception/lidar_geo	LF1:PC/S	LF2:FPC/S	4500	4670
/perception/lidar_voxel	LF2:FPC/S	LF3:FPC/S	2200	2250
/perception/lidar_ground	LF3:FPC/S	LF4:FPC/S	1280	1350
/perception/lidar_track	LF4:FPC/S	LF5:FPC/S	55.36	65.4
/perception/lidar_ground	LF3:FPC/S	LF6:PC/S	1280	1350
/perception/lidar_crop	LF6:FPC/S	LF7:NPC/S	1220	1300
/perception/lidar	LF5:FPC/S	LCL:FPC/S	50.00	70.23
/perception/lidar	LF5:FPC/S	LNN:FPC/S	50.00	70.23
/perception/lidar_negative_track	LF7:NPC/S	LiLo:NPC/S	750	850
/sensor/radar	RADAR	RF1:PC/S	290	310
/perception/radar_negative_track	RF1:FPC/S	RaLo:NPC/S	210	220
/perception/radar	RF1:FPC/S	RCL:FPC/S	67.25	89.44
/state/radar_odometry	RaLo:NPC/S	SE:RaLo/S	0.73	0.73
/state/lidar_odometry	LiLo:NPC/S	SE:LiLo/S	0.73	0.73
/sensor/imu	IMU	SE:IMU/S	0.32	0.32
/kistler/cor	Correvit	SE:CO/S	0.73	0.73
/perception/detect_obj_cluster	LCL:FPC/S	OT:LCL/S	0.79	1.60
/perception/detect_obj_lidar	LNN:PC/S	OT:LNN/S	0.77	0.81
/perception/detect_obj_radar	RCL:FPC/S	OT:RCL/S	0.77	0.81
/state/odometry	SE:KF/T	OT:SE/S	0.73	0.73
/planning/predicted_objects	OT:KF/T	SP:OT/S	3.36	3.36
/state/odometry	SE:KF/T	SP:SE/S	0.73	0.73
/planning/global_path	BP:GP/T	SP:BP/S	15.73	15.73
/state/odometry	SE:KF/T	BP:SE/S	0.73	0.73
/planning/target_trajectory/xy	SP:TP/T	TC:TP/S	7.63	7.63
/state/odometry	SE:KF/T	TC:SE/S	0.73	0.73
/control/control_command	TC:CO/T	SC:CO/S	0.05	0.05
/control/control_command	TC:CO/T	LC:CO/S	0.05	0.05
/control/lateral_request	SC:CO/S	VG:SC/S	0.02	0.02
/control/longitudinal_request	LC:CO/S	VG:LC/S	0.06	0.06
/actuation/lateral	VG:SC/S	VI:SC/S	0.02	0.02
/actuation/longitudinal	VG:LC/S	VI:LC/S	0.06	0.06
/can/longitudinal	VI:LC/S	CAN0	-	-
/can/lateral	VI:SC/S	CAN1	-	-

TABLE 4. Priorization of individual callback chains.

Chain	Prio	Description
(1) LiDAR →{LF} →LiLo →SE	1	Since the system is designed for high-speed driving, reliable localization is required for stable control performance.
(2) SE →TC →SC →VG →VI	1	Lateral controller to calculate the steering angle of the vehicle.
(3) SE →TC →LC →VG →VI	1	Longitudinal controller to calculate the acceleration and braking of the vehicle.
(4) Correvit →SE	1	Measurement of the longitudinal and lateral velocity of the vehicle.
(5) IMU →SE	1	Measurement of vehicles acceleration and turn rate.
(6) RADAR →{RF} →RaLo →SE	2	Redundant localization pipeline that stabilizes and supports LiDAR odometry.
(7) LiDAR →{LF} →LNN →OT	3	Used as main object recognition because it can estimate the heading of the recognized vehicles. It can open corresponding objects in tracking.
(8) OT →SP →TC	3	Object behavior prediction with subsequent path planning and tracking.
(9) LiDAR →{LF} →LCL →OT	4	Secondary object detection because it cannot estimate a heading. It can open corresponding objects in tracking.
(10) SE →OT	5	Using vehicles state estimation in order to better predict others vehicles.
(11) RADAR →{RF} →RCL →OT	5	Assists with 360° sensor coverage. Cannot create objects in tracking as object detection is not as consistent as LiDAR. Assists in cases where vehicles are out of sight of the LiDAR.
(12) SE →BP →SP	6	Sends global path of the vehicle that is valid for a longer time horizon (5 s).
(13) SE →SP	6	Update of vehicle ego pose in order to calculate the next trajectory sampling.

LiDAR, RADAR, IMU, and Correvit are sensor interfaces or respectively sensor drivers. It is noteworthy that LiLo and RaLo are the most computationally intensive nodes, while state estimation (SE) and trajectory control (TC) have the lowest execution times. As illustrated in Table 3, the messages within the systems and between the respective publishers and subscribers are defined, along with their message sizes. Notably, in the case of perception, message sizes are particularly large and exhibit high volatility due to a changing point cloud size. In an ideal optimization scenario, each IPC configuration should be measured in every possible combination in order to parametrize the communication latencies in the optimization. As previously outlined in Section VI-B4, an approximation method is employed. To characterize the communication behavior of the HPC

TABLE 5. Optimization results generated by Gurobi. Objective and upper bound are in s.

Configuration				Executors	Objective	Upper Bound
Weight	Container	Timer	IPC			
Equal	False	Static	False	26	4.866	4.866
Equal	False	Static	True	19	4.398	4.398
Equal	False	Discrete	False	24	4.816	4.816
Equal	False	Discrete	True	19	4.341	4.341
Equal	True	Static	False	12	5.837	5.837
Equal	True	Static	True	12	5.178	5.178
Equal	True	Discrete	False	12	5.781	5.781
Equal	True	Discrete	True	12	5.122	5.122
Linear	False	Static	False	25	2.380	4.866
Linear	False	Static	True	19	2.197	4.398
Linear	False	Discrete	False	26	2.372	4.816
Linear	False	Discrete	True	19	2.182	4.341
Linear	True	Static	False	12	2.833	5.845
Linear	True	Static	True	12	2.520	5.325
Linear	True	Discrete	False	12	2.818	5.788
Linear	True	Discrete	True	12	2.505	5.268
Log	False	Static	False	26	1.248e-3	4.913
Log	False	Static	True	20	1.207e-3	5.034
Log	False	Discrete	False	25	1.248e-3	5.296
Log	False	Discrete	True	20	1.200e-3	4.949
Log	True	Static	False	12	1.379e-3	9.857
Log	True	Static	True	12	1.253e-3	7.862
Log	True	Discrete	False	12	1.378e-3	9.805
Log	True	Discrete	True	12	1.246e-3	7.386

platform using IPC communication, a system identification has been performed. A workload is executed on the system consisting of two callbacks communicating with a message of variable size. The measured communication latencies are shown in Fig. 10 in the Appendix B. To parametrize the optimization model, the worst-case latencies for the callback duration and communication latencies are needed.

C. OPTIMIZATION EVALUATION

As delineated in the preceding chapter, the worst-case execution times for the callbacks and communications are imperative for the parametrization of the optimization. A challenge emerges from the system's composition of multiple callback sequences. As elucidated in Section VI-B1, we employed a prioritization logic for the potential chains, which is delineated in Table 4. The total number of configurations under consideration was 24, and these will be discussed subsequently. Each configuration's optimization was parametrized and the configurations were based on the *weighting factor* (equal, linear, and logarithmic), *container optimization* (false vs. true), *timer* (static vs. discrete), and *IPC* (false vs. true), The upper bound represents the theoretical worst case scenario, which has been calculated as the sum of all chains. The optimization results generated by Gurobi [75] can be derived from Table 5. Beginning with the *container setup*, it becomes evident that, for all weighting schemes, the upper bound is considerably smaller for the *Container=false* scenario. This can be attributed to the optimization algorithms' assumption of one core per executor. This optimization outcome aligns with the approach described in [8], and represents the current state of research in this field. However, it is important to note that this method is not universally applicable, as the number of resulting executors exceeds the limit of available cores in our system. Therefore, it is necessary to set the container setting to true, thereby enabling to group executors together.

Initially, we consider equal weighting of the callback chains, and for the *timer*, we observe a marginal smaller upper bound for the discrete setup (e.g., 4.82 s vs. 4.87 s). This pattern was also, as expected, visible for linear and logarithmic weighting.

Comparing the *IPC configuration*, we noted a clear improvement for IPC configured to allow for direct communication with each other, with an improvement of approximately 500 ms for the equal and linear weighting scheme. The values were slightly lower for the logarithmic weighting scheme.

In light of these considerations, it is evident that for all weighting schemes, container optimization remains unestablished. We employ a discrete timer and facilitate direct communication. The resulting upper bounds are 5.12 s for equal weighting, 5.27 s for linear weighting, and 7.38 s for logarithmic weighting.

The system configurations for each weighting function were derived from the selected configurations for the three weighting schemes, as illustrated in Table 6. This process resulted in a list of 12 executors, along with the nodes allocated per weighting factor. In some cases, the executor assignments remained unaltered, while in others, there was a substantial difference. For instance, identical nodes and priorities were achieved for the executors 1-3, 5-6, 9, and 12. At the same time, we observed clear allocation differences for other executors in terms of node composition and the priority in which they are scheduled. Regarding task policy, for all Python nodes (Executors 9 and 10) the task scheduling prio changed to subscription-first. For the C++ nodes with timers (Executor 11 and 12) in the baseline configuration, subscription-first was already instrumented, therefore there

TABLE 6. Executor configuration comparison after the optimization.

Executor	Weight	Nodes	Task Policy	Util.
1	Equal	RaDAR	-	0.189
	Linear	RaDAR	-	0.189
	Log	RaDAR	-	0.189
2	Equal	LiDAR	-	0.448
	Linear	LiDAR	-	0.448
	Log	LiDAR	-	0.448
3	Equal	LF2, LF1	-	0.388
	Linear	LF2, LF1	-	0.388
	Log	LF2, LF1	-	0.388
4	Equal	LF5, LF6, LF4, LF3	-	0.327
	Linear	LF6, LF3, IMU	-	1.071
	Log	LF6, LF3	-	0.157
5	Equal	LiLo, LF7	-	1.984
	Linear	LiLo, LF7	-	1.984
	Log	LiLo, LF7	-	1.984
6	Equal	LNN	-	0.425
	Linear	LNN	-	0.425
	Log	LNN	-	0.425
7	Equal	LCL, RaLo	-	4.164
	Linear	RCL, RaLo, RF1	-	3.949
	Log	RaLo, LF5, RCL, LCL, LF4, RF1	-	4.836
8	Equal	RCL, RF1	-	2.304
	Linear	LF5, LCL, LF4	-	0.888
	Log	IMU, Correvit	-	2.377
9	Equal	OT	$sub > tmr$	6.559
	Linear	OT	$sub > tmr$	6.559
	Log	OT	$sub > tmr$	6.559
10	Equal	SP, BP	$sub > tmr$	7.116
	Linear	SP, BP	$sub > tmr$	7.116
	Log	BP, SP	$sub > tmr$	7.116
11	Equal	IMU, Correvit, SE	$sub > tmr$	17.910
	Linear	Corevit, SE	$sub > tmr$	17.685
	Log	SE	$sub > tmr$	17.618
12	Equal	VI, TC, SC, LC, VG	$sub > tmr$	1.077
	Linear	VI, TC, SC, LC, VG	$sub > tmr$	1.077
	Log	VI, TC, SC, LC, VG	$sub > tmr$	1.077

is no change in prioritization. Focusing on timer periods, our optimization framework modified these timer configurations. For instance, the object tracking timer callback now runs at an interval of 10 ms, which is an decrease from the original 50 ms baseline. For the sampling planner the optimization gives a timer period of also 10 ms instead of 50 ms. For the relatively slow behavior planner there is a decrease from 100 ms to 50 ms. For the performance relevant tracking controller there is a decrease from 10 ms to 6.67 ms. The state estimation runs, after the optimization, with a period of 3.33 ms instead of 10 ms. In order to do so without changing the algorithm quality, the internal time constants for the Kalman filter need to be adapted to match the new period.

D. END-TO-END LATENCY EVALUATION

Subsequent to executing the optimized configurations, the statistical descriptives (see Appendix C) for our individual callback sequences were obtained. For the 13 chains, we obtained min and max values, mean, median, and P99, as well as our theoretical upper bound maximum that has been calculated from the optimization. In addition, the baseline measurement was incorporated into the analysis. As previously emphasized, the maximum values represent the focal point of our investigation as this is our optimization target. Fig. 8 shows the cumulative probability distribution of the main callback chains. The dotted lines represent the maximum value for the respective configurations. Initially, a comparison was made between the maximum values and the upper bound as it is imperative that the former always lies beneath the latter. Our findings substantiate this supposition, as the upper bound holds for every chain and configuration. Additionally, it is evident that for chains with identical configurations, e.g., chains 2 or 3, a similar pattern is observed in the maximum values, with only minor deviations.

Initially, the first chain exhibited configurations that consistently outperforms the baseline measurement (maximum = 553.888 ms). Notably, the logarithmic weighting configuration attained a maximum value of 437.839 ms. The second chain, which has the same node executor composition for all weighting schemes, achieves maximum values ranging from 8.670 ms to 8.764 ms, providing, as with the upper bound, very comparable results. It is evident that there is a substantial enhancement in all optimized configurations, with an increase of about 4 ms for each configuration. The differences in the mean values are negligible. The third chain demonstrates a similar trend, with the lowest mean value of 9.113 ms attained by equal weighting. The differences between the configurations are statistically insignificant, while the deviations from the baseline are statistically significant. The sixth cause-effect chain exhibits a maximum value of 495.955 ms, attained by linear weighting, while maintaining the lowest mean value. In this case, there are minimal differences, with only two configurations demonstrating marginal superiority. Equal weighting demonstrates optimal performance for the seventh chain, with a maximum value of 306.327 ms. This scenario reiterates the finding that the logarithmic weighting scheme exhibits suboptimal performance in comparison to the baseline measurement. The remaining configurations yield superior performance with medium changes. The subsequent chain once again exhibits a clear outperformance relative to its theoretical maximum. All configurations fall below the baseline measurement, but the differences within the configurations are negligible, ranging from approximately 125 ms to 130 ms. The next value chain demonstrates a substantial enhancement with the linear weighted configuration, achieving a maximum value of 295.789 ms, in contrast to the baseline measurement of 426.502 ms. However, it is important to note that this significant discrepancy in mean values is not observed. Chain 11 displays a comparable pattern, with logarithmic weighting yielding a higher maximum value, as reflected in the mean values. Equal weighting demonstrates optimal performance in this scenario, with a mean value of 267.107 ms (baseline = 292.118 ms). For chain 12 a more analogous distribution of results is observed in the final of the primary chains. In this case, the maximum baseline is 184.879 ms, and equal weighting is once again the optimal strategy. This outcome aligns with the upper bound condition.

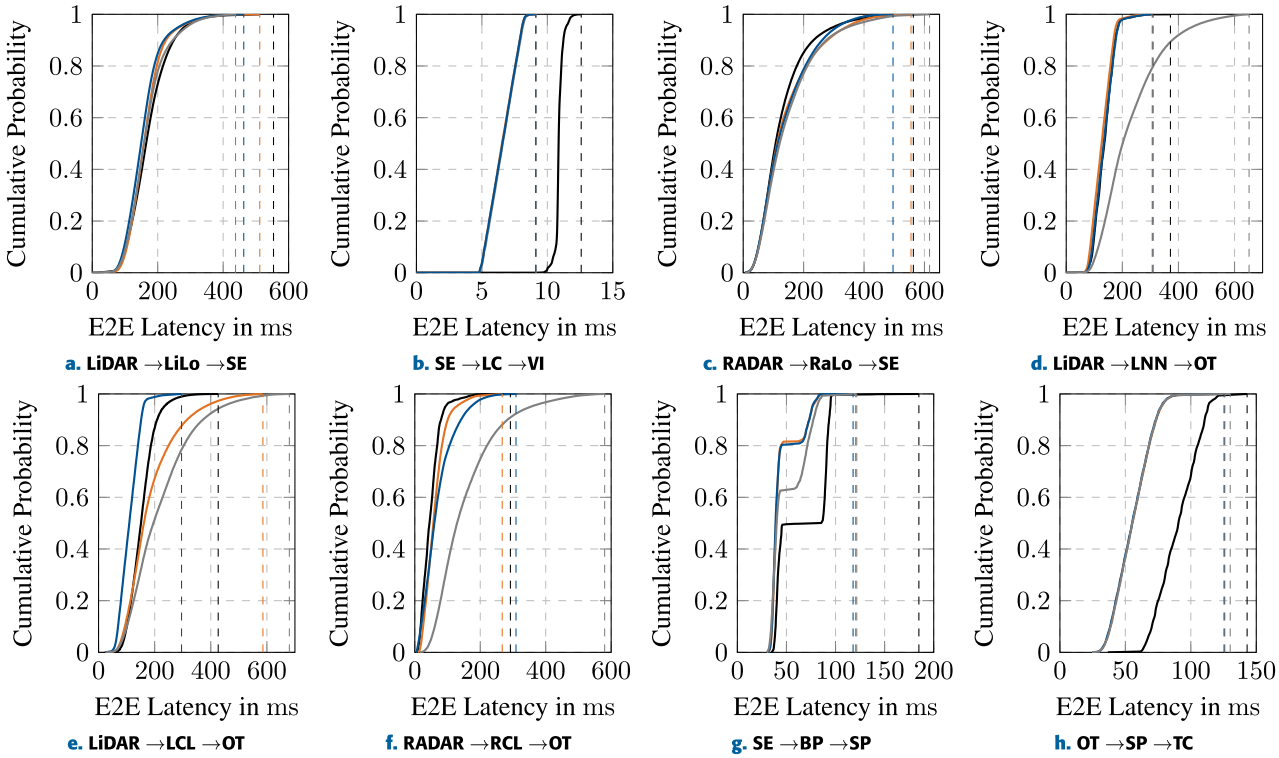


FIGURE 8. Figures of the cumulative probability distribution for the end-to-end latency of the main callback chains. The maximum values of the respective measurement are depicted as dotted line. *Baseline = black, equal = orange, linear = blue, logarithmic = gray.*

The salient points can be enumerated as such: It is noteworthy that the maximum values always fall below the theoretical maximum, which is defined as the upper bound. It is evident that configurations do not invariably outperform the baseline measurement; however, for each value chain, there exists a configuration that exhibits superior performance, albeit occasionally modest. This phenomenon can be attributed to the implementation of disparate weighting schemes and the application of our own prioritization logic to the value chain. Consequently, different approaches may yield varying degrees of success depending on the specific characteristics of the chain in question.

Adding up the measured maximum values of all chains, the baseline results in a total of 2.880 s. Optimization with equal weighting yields 2.598 s, which is an improvement of 9.79 %. The biggest improvement is seen with linear weighting. The total runtime here is 2.233 s, which is an improvement of 22.47 % over the maximum end-to-end latency. On the other hand, the logarithmic weighting shows a decrease in the overall system. With a total runtime of 3.353 s, this is 16.42 % more than the baseline. This is partly due to executor usage, which is not taken into account. Executor 7 in particular causes the latencies of several chains to largely deteriorate.

E. IPC APPROXIMATION

This section validates IPC communication by comparing the baseline and our optimized configurations to assess performance improvements. To this end, a composition with IPC was selected for investigation: LF1 and LF2 filter, representing the largest message in the software stack (4670 kB). It is crucial to note that the aforementioned case is analogous across all three optimization configurations, so also the consistency is observable. As illustrated in Fig. 9, the baseline configuration (with DDS) demonstrates a maximum value of 11.997 ms. This value is used for as communication cost for the optimization. It is evident that all optimized configurations demonstrate a substantial enhancement, with a max reduction to approximately 2.396 ms, which is considerably smaller and approaches our

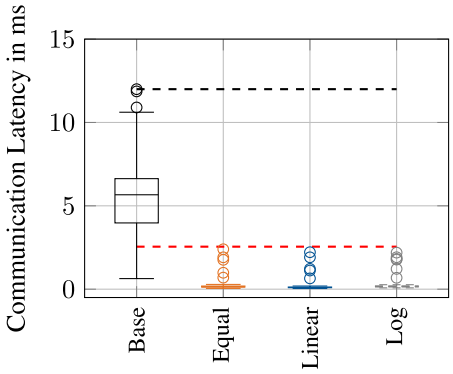


FIGURE 9. Validation of the IPC approximation method using the LF1 and LF2 filter. The black dashed line is the maximum communication cost in the baseline. The red dashed line is the approximated IPC value.

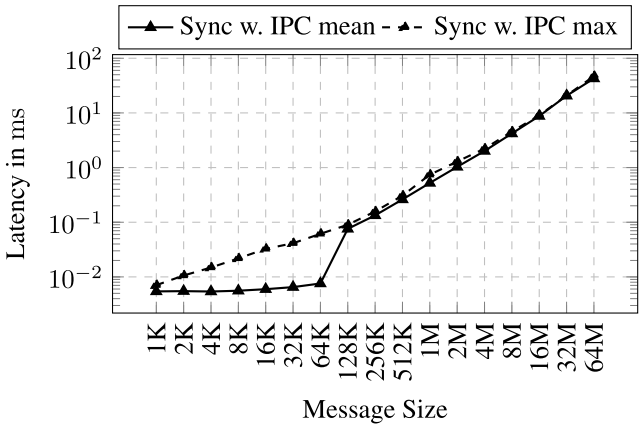


FIGURE 10. Latency values of IPC communication between two nodes with variable message sizes.

estimated maximum value of 2.550 ms. As visible in the figures, the measured latency is almost consistent with the approximated value. IPC communication for this example is an improvement of almost 20 % for the maximum observed measurement. Therefore, it is a time-saving way to use this method for containerized systems.

F. JITTER ANALYSIS

Jitter analysis was performed to validate the precision and reliability of the timers after the optimization. See Appendix D for an overview of the values achieved, comparing the baseline measurement with the optimized configuration. Beginning with state estimation and control, it is evident that both hold their periods perfectly, exhibiting almost no variance. The timer period for state estimation is consistent with the optimized configuration, measuring at 3.33 ms. Control analogues exhibited a consistent period of 6.67 ms. However, object tracking revealed a high degree of variance in the baseline measurement, attributable to the presence of two executors on the same core, specifically the behavior planner. Consequently, jitter was observed to be high when the behavior planner exhibited elevated execution times, thereby blocking the timer from execution. Notably, the optimized configurations demonstrated a substantial enhancement, with the object tracking timer maintaining a frequency of 10 ms with minimal variability. Conversely, the sampling planner demonstrates adequate performance in the baseline measurement with minimal variability. However, for the optimized configurations, the targeted frequency of 10 ms is not attained, as the mean value recorded was 40.59 ms, with a range between 30.89 ms and 50.44 ms. This results in a secondary execution of a node within the executor, which hinders the operation of the timer. Finally, the behavior planner maintains an average frequency of 50 ms which is the target period, but exhibits greater variance.

VIII. DISCUSSION

The objective of our research was to develop and implement an optimization framework capable of enhancing the performance of ROS 2 systems with respect to timing and end-to-end latency. We have created an optimization framework that optimizes container, timer, and IPC. The integration of diverse weighting schemes has demonstrated the potential for system optimization with multiple interacting callback sequences. Our experimental findings have revealed that, under equal and linear weighting conditions, we can achieve an outperformance of the baseline measurement by 9.79 % and 22.47 %. However, the application of logarithmic weighting has been observed to result in underperformance due to the presence of individual chains with significantly higher runtimes. A more thorough investigation through jitter analysis revealed that, for specific nodes, the objectives established by the optimization process were successfully attained. However, it is anticipated that substantially higher frequencies could be attainable. Nevertheless, it is imperative to exercise caution, as the adaptation of time constants in Kalman filters and integrators is necessary to achieve further enhancements. With the timers we used, we still observed a stable performance in the driving dynamic relevant nodes. In instances where two nodes are operating on a single executor and possess elevated maximum values, attaining low timer frequencies becomes impracticable. Nevertheless, even in scenarios where the targeted frequency is not attained, a reduction in the max and mean end-to-end latency is observed when compared to the baseline measurement. Additionally, the efficacy of IPC communication enhancement has been demonstrated. However, it should be noted that achieving a more substantial enhancement would necessitate the identification and optimization of all IPC communication chains, as it was the case in [8]. However, our approximation is a valid assumption. We validated the accuracy and therefore it is a more time efficient way than implementing, testing, and measuring each possible IPC combination in the system. It is also important to acknowledge that the impact is negligible as the chains become more complex, particularly when callbacks with high runtimes are incorporated, such as LiLo or RaLo. The efficacy of the optimization framework would be significantly enhanced if callbacks possessed substantially lower maximum runtimes, allowing for further optimization. A closer look at the optimization procedure reveals that it does not fully align with the fundamental principles of container architecture development. For instance, placing IMU and perception within a single container does not reflect a practical approach. However, it is evident that, under our linear weighting scheme, the optimization framework tends to group closely related tasks within a single container. This behavior is similar to the equal weighting scheme, but with a higher propensity to group containers with different functions. A salient issue that emerges in this context pertains to safety criticality, particularly from a practical perspective. In real-world testing situations, the grouping of nodes within a single executor can potentially lead to a hazardous scenario wherein the failure of a single node results in the failure of the entire executor. To mitigate this risk, it would be prudent to prohibit the composition of nodes that should not be grouped

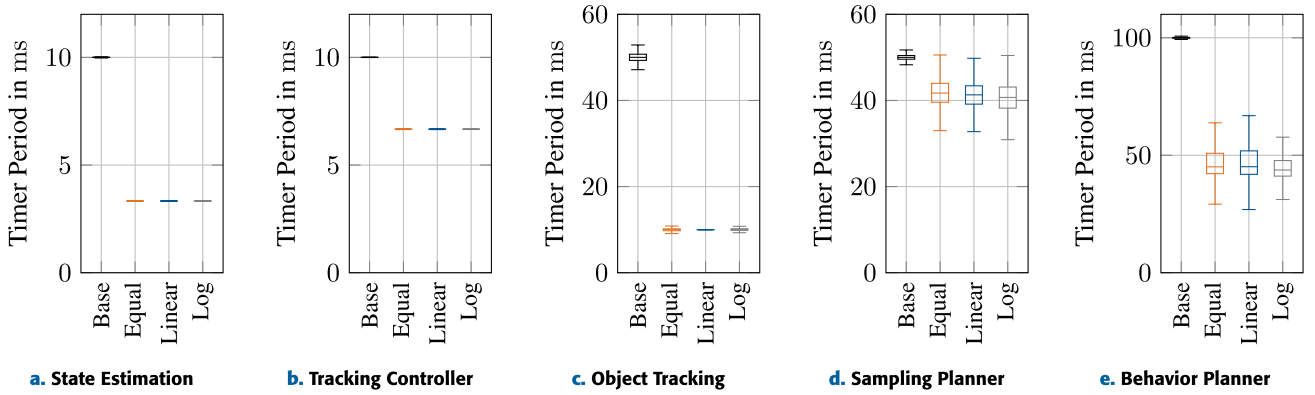


FIGURE 11. Jitter analysis for the different timer callbacks.

together. This safety prioritization approach, for instance, could be advantageous in scenarios involving redundant sensors, such as the IMU, by ensuring their placement on individual cores. In addition, the executor utilization was not taken into consideration. Such an integration would be beneficial; however, within the system, a problem was identified of high maximum values compared to the mean, which resulted in utilization over a value of 1 without optimization. This would have meant that no additional nodes would have been placed on those executors. This problem was most evident in the logarithmic weighting approach. In this approach, a situation arises in which nearly all perception nodes are concentrated in a single executor, resulting in elevated maximum values. Furthermore, the optimization model did not incorporate all subscriptions and publishers due to constraints that had to be imposed. Our analysis focused exclusively on system-relevant cause-effect chains while excluding certain service function chains.

IX. CONCLUSION

Within our work we were able to show that an optimization strategy can lead to significant performance increases. Therefore, this research highlights practical recommendations as well as opens up further research potential. Overall, we recommend that the optimization of microservice architectures should be designed at an early stage of development, as it strongly influences the software architecture building process. If all dependencies would be handled in a base image, which contradicts the idea of microservice containerization, this would support the timing optimization. A balanced view between performance and software architecture is required. Furthermore, we recommend to develop algorithms directly as ROS 2 components to make runtime compositions. Another key takeaway is the goal of having one executor and one container per core. Additionally, we made first attempts to investigate the impact of latency improvement on performance. Here we saw a correlation with our configurations, which would be a topic of interest as it is of high importance for racing scenarios. This would enable a better understanding in how much latency costs performance in high speed scenarios. Future work should focus on OS-level scheduling and utilization, as we were limited here due to the software stack we used. Additionally, future work could explore and compare alternative optimization approaches such as genetic algorithms to assess their effectiveness and suitability for scheduling in ROS 2-based containerized execution environments. Furthermore, an improvement would be to move from a deterministic to a probabilistic approach, using the distribution instead of the maximum execution time.

TABLE 7. List of variables and descriptions.

Variable	Description
$x_e \in \{0,1\},\ e \in \mathcal{E}$	Executor task policy (0 = *tmr* first, 1 = *sub* first)
$x_{n,e} \in \{0,1\},\ n \in \mathcal{N}, e \in \mathcal{E}$	Node-executor assignment (1 = assigned, 0 = not assigned)
$x_{t,n} \in \{0,1\},\ t \in \mathcal{T}, n \in \mathcal{N}$	Task-node assignment (1 = assigned, 0 = not assigned)
$x_{t,e} \in \{0,1\},\ t \in \mathcal{T}, e \in \mathcal{E}$	Task-executor assignment (1 = assigned, 0 = not assigned)
$x_t \in \{0,1\},\ t \in \mathcal{T}$	Task policy
$x_{t,t'} \in \{0,1\},\ t,t' \in \mathcal{T}$	Equality between task executors
$c_e \in \mathbb{Z},\ e \in \mathcal{E}$	Task count per executor
$x_{n,n'} \in \{0,1\},\ n,n' \in \mathcal{N}$	Equality between node executors
$p_n \in \mathbb{Z},\ n \in \mathcal{N}$	Node priorities
$p_t \in \mathbb{Z},\ t \in \mathcal{T}$	Task priorities
$t_i \in \mathbb{R},\ i \in \mathcal{T}$	Timer periods
$w_t \in \mathbb{R},\ t \in \mathcal{T}$	Worst-case execution times (WCET)
$c_{i,j} \in \mathbb{R},\ i \in \mathcal{T}, j \in \mathcal{P}_i$	Communication costs
$s_t \in \mathbb{R},\ t \in \mathcal{T}$	WCET including communication costs
$s_e \in \mathbb{R},\ e \in \mathcal{E}$	Total WCET per executor
$s_{t,e} \in \mathbb{R},\ t \in \mathcal{T}$	WCET sum for task executor
$h_t \in \mathbb{R},\ t \in \mathcal{T}$	Higher-priority task WCET sums
$l_t \in \mathbb{R},\ t \in \mathcal{T}$	Lower-priority task WCET sums

APPENDIX A
OPTIMIZATION ALGORITHM

Sets:

- $\mathcal{N}$: Set of nodes.
- $\mathcal{T}$: Set of tasks.
- $\mathcal{E}$: Set of executors.
- $\mathcal{C}$: Set of chains.
- $\mathcal{P}_t$: Set of publishers for task t.

TABLE 8. List of constraints and descriptions.

Constraint	Description
$\sum_{e\in\mathcal{E}} x_{n,e} = 1,\ \forall n \in \mathcal{N}$	Each node assigned to exactly one executor
$x_{n,e} + x_{n',e} \leq 1,\ \forall n \in \mathcal{N}_{cpp}, n' \in \mathcal{N}_{py}, e \in \mathcal{E}$	Language isolation
$t_i = t_i^{\text{static}},\ \forall i \in \mathcal{T}$	Static timer periods
$\sum_{n\in\mathcal{N}} x_{t,n} = 1,\ \forall t \in \mathcal{T}$	Each task assigned to one node
$\sum_{e\in\mathcal{E}} x_{t,e} = 1,\ \forall t \in \mathcal{T}$	Each task assigned to one executor
$x_{t,e} = \sum_{n\in\mathcal{N}} x_{t,n} \cdot x_{n,e},\ \forall t \in \mathcal{T}, e \in \mathcal{E}$	Task-Executor consistency
$x_t = \sum_{e\in\mathcal{E}} x_{t,e} \cdot x_e,\ \forall t \in \mathcal{T}$	Task policy propagation
$x_{t,t'} = \sum_{e\in\mathcal{E}} x_{t,e} \cdot x_{t',e},\ \forall t, t' \in \mathcal{T}$	Task equality
$c_e = \sum_{t\in\mathcal{T}} x_{t,e},\ \forall e \in \mathcal{E}$	Task count per executor
$p_n \geq p_{n'} + 1,\ \forall n, n' \in \mathcal{N} : x_{n,n'} = 1$	Node priority order
$p_t \geq p_{t'} + 1,\ \forall t, t' \in \mathcal{T} : x_{t,t'} = 1$	Task priority order
$w_t = p_{999}(t),\ \forall t \in \mathcal{T}$	Task WCET
$s_t = w_t + \sum_{j\in\mathcal{P}_t} c_{t,j},\ \forall t \in \mathcal{T}$	Task WCET sum
$s_e = \sum_{t\in\mathcal{T}} s_t \cdot x_{t,e},\ \forall e \in \mathcal{E}$	Executor WCET sum
$s_{t,e} = \sum_{e\in\mathcal{E}} s_e \cdot x_{t,e},\ \forall t \in \mathcal{T}$	Task executor WCET sum
$h_t = \sum_{t'\in\mathcal{T}} s_{t'} \cdot x_{t',t},\ \forall t \in \mathcal{T}$	Higher priority WCET sum
$l_t = \sum_{t'\in\mathcal{T}} s_{t'} \cdot x_{t,t'},\ \forall t \in \mathcal{T}$	Lower priority WCET sum

TABLE 9. Statistical descriptive for the individual chains for baseline and all optimization configurations. Values are in ms.

Chain	Config	Min	Mean	Median	P99	Max	Upper Bound
(1) LiDAR →{LF} →LiLo →SE	Baseline	15.980	172.344	164.623	358.595	553.888	-
	Equal	25.772	164.600	157.784	354.195	511.839	940.18
	Linear	19.448	156.657	149.365	350.227	462.935	915.91
	Log	18.280	167.853	158.452	334.013	437.839	**899.84**
(2) SE →TC →SC →VG →VI	Baseline	8.707	11.030	11.043	11.978	12.641	-
	Equal	4.720	6.546	6.537	8.284	8.670	28.63
	Linear	4.735	6.556	6.561	8.279	8.725	28.63
	Log	4.769	6.595	6.593	8.337	8.764	28.63
(3) SE →TC →LC →VG →VI	Baseline	9.185	10.899	10.861	11.832	12.588	-
	Equal	4.732	6.553	6.538	8.301	9.114	28.66
	Linear	4.748	6.563	6.564	8.292	9.113	28.66
	Log	4.779	6.600	6.596	8.346	9.151	28.66
(4) Correvit →SE	Baseline	0.153	2.448	1.947	5.660	9.677	-
	Equal	0.152	2.410	2.446	4.939	8.258	125.51
	Linear	0.155	2.353	2.297	4.502	7.486	121.91
	Log	0.153	2.156	2.159	4.138	4.512	**83.29**
(5) IMU →SE	Baseline	0.148	2.212	0.581	5.375	10.936	-
	Equal	0.150	2.137	2.045	4.678	7.925	127.99
	Linear	0.149	2.462	2.331	4.409	6.703	90.13
	Log	0.153	2.680	2.679	5.190	5.785	**79.99**
(6) RADAR →{RF} →RaLo →SE	Baseline	6.493	128.838	107.264	430.387	563.248	-
	Equal	6.266	141.518	112.145	437.674	555.494	861.26
	Linear	5.778	138.122	114.388	392.917	495.955	**837.28**
	Log	5.819	147.126	120.611	477.719	616.814	982.92
(7) LiDAR →{LF} →LNN →OT	Baseline	49.230	135.976	135.819	238.831	371.473	-
	Equal	43.257	127.528	126.192	222.722	306.327	**478.95**
	Linear	45.474	135.035	133.747	236.134	309.473	583.19
	Log	47.353	229.174	201.145	576.976	651.839	1313.91
(8) OT →SP →TC	Baseline	36.646	90.400	90.488	120.285	143.002	-
	Equal	24.990	55.913	55.321	85.488	126.003	**396.00**
	Linear	24.885	55.634	55.198	84.789	125.386	**396.00**
	Log	24.485	55.866	55.226	84.886	130.086	397.64
(9) LiDAR →{LF} →LCL →OT	Baseline	22.469	154.394	151.242	296.155	426.502	-
	Equal	22.864	186.581	161.639	489.952	585.874	752.65
	Linear	23.128	110.864	109.529	206.836	295.789	**664.81**
	Log	25.229	220.015	195.154	566.904	680.122	1666.37
(10) SE →OT	Baseline	4.580	11.426	10.682	22.282	177.923	-
	Equal	4.221	9.287	9.296	12.424	15.529	190.19
	Linear	2.579	6.546	6.528	9.696	12.269	**188.39**
	Log	3.576	10.064	10.076	13.007	30.238	255.28
(11) RADAR →{RF} →RCL →OT	Baseline	5.748	48.692	44.869	161.847	292.118	-
	Equal	9.346	61.934	56.605	183.445	267.107	**418.31**
	Linear	3.418	71.072	59.714	225.674	309.172	645.35
	Log	15.126	155.987	129.286	488.211	580.088	1090.97
(12) SE →BP →SP	Baseline	34.511	66.419	85.398	95.545	184.879	-
	Equal	31.028	45.107	39.246	83.005	120.386	428.03
	Linear	30.360	45.127	38.912	82.417	117.873	**426.23**
	Log	29.855	50.555	40.064	86.052	121.606	428.59
(13) SE →SP	Baseline	21.174	31.158	30.803	35.712	121.315	-
	Equal	18.940	26.323	26.101	32.201	75.407	218.40
	Linear	18.952	26.386	26.088	31.824	71.882	**216.59**
	Log	19.486	27.531	27.191	35.890	76.458	218.53

APPENDIX B
IPC MEASUREMENT

See Figure 10.

APPENDIX C
END-TO-END LATENCY

See Table 9.

APPENDIX D
JITTER ANALYSIS

See Figure 11.

ACKNOWLEDGMENT

Tobias Betz, as the first author, was the initiator of the research idea and is responsible for the presented concept and implementation. He was involved in all stages of the development and implementation. Harun Teper contributed to the model description and optimization implementation. Dominic Ebner contributed to the closed-loop simulation environment and overall software development. Maximilian Leitenstern contributed to the localization setup in the simulation and evaluation. Simon Sagmeister contributed to the parametrization, implementation, and adoption of the control algorithm. Marcel Weinmann contributed to the parametrization and adoption of the state estimation. Jian-Jia Chen contributed to the conception of the research project and revised the article critically for important intellectual content. Markus Lienkamp contributed to the conception of the research project and revised the article critically for important intellectual content. He gave final approval of the version to be published and agrees to all aspects of the work. As a guarantor, he accepts responsibility for the overall integrity of the article. The authors want to thank the entire TUM Autonomous Motorsport Team for contributing to the overall software stack over the last years.

REFERENCES

[1] T. Betz, P. Karle, F. Werner, and J. Betz, "An analysis of software latency for a high-speed autonomous race car—A case study in the indy autonomous challenge," *SAE Int. J. Connected Automated Vehicles*, vol. 6, no. 3, pp. 283–296, Feb. 2023.

[2] *Autosar Specification of Timing Extensions 4.4.0*, AUTOSAR, Munich, Germany, 2018.

[3] D. Casini, T. Blaß, I. Lütkebohle, and B. B. Brandenburg, "Response-time analysis of ROS 2 processing chains under reservation-based scheduling," in *Proc. 31st Euromicro Conf. Real-Time Syst.*, vol. 5, Jan. 2019, p. 23.

[4] T. Blaß, D. Casini, S. Bozhko, and B. B. Brandenburg, "A ROS 2 response-time analysis exploiting starvation freedom and execution-time variance," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2021, pp. 41–53.

[5] T. Blass, A. Hamann, R. Lange, D. Ziegenbein, and B. B. Brandenburg, "Automatic latency management for ROS 2: Benefits, challenges, and open problems," in *Proc. IEEE 27th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2021, pp. 264–277.

[6] H. Teper, M. Günzel, N. Ueter, G. von der Brüggen, and J.-J. Chen, "End-to-end timing analysis in ROS2," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2022, pp. 53–65.

[7] M. Günzel, H. Teper, K.-H. Chen, G. von der Brüggen, and J.-J. Chen, "On the equivalence of maximum reaction time and maximum data age for cause-effect chains, "On the equivalence of maximum reaction time and maximum data age for cause-effect chains," in *Proc. 35th Euromicro Conf. Real-Time Syst. (ECRTS)* (Leibniz International Proceedings in Informatics (LIPIcs)), vol. 262. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 2023, pp. 10:1–10:22. [Online]. Available: https://doi.org/10.4230/LIPIcs.ECRTS.2023.10

[8] H. Teper, T. Betz, M. Günzel, D. Ebner, G. von der Brüggen, J. Betz, and J.-J. Chen, "End-to-end timing analysis and optimization of multi-executor ROS 2 systems," in *Proc. IEEE 30th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2024, pp. 212–224.

[9] J. Betz, T. Betz, F. Fent, M. Geisslinger, A. Heilmeier, L. Hermansdorfer, T. Herrmann, S. Huch, P. Karle, M. Lienkamp, B. Lohmann, F. Nobis, L. Ögretmen, M. Rowold, F. Sauerbeck, T. Stahl, R. Trauth, F. Werner, and A. Wischnewski, "TUM autonomous motorsport: An autonomous racing software for the indy autonomous challenge," *J. Field Robot.*, vol. 40, no. 4, pp. 783–809, Jun. 2023.

[10] IAC. (2021). *IAC: Indy Autonomous Challenge*. [Online]. Available: https://www.indyautonomouschallenge.com
[11] Aspire. (2024). *A2RL: Abu Dhabi Autonomous Racing League*. [Online]. Available: https://www.a2rl.io
[12] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, May 2022, Art. no. eabm6074.
[13] M. M. Scheunemann and S. G. V. Dijk, "ROS 2 for RoboCup," in *Robot World Cup*, 2019, pp. 429–438.
[14] P. Phueakthong and J. Varagul, "A development of mobile robot based on ROS2 for navigation application," in *Proc. Int. Electron. Symp. (IES)*, Sep. 2021, pp. 517–520.
[15] L. Bianchi, D. Carnevale, F. Del Frate, R. Masocco, S. Mattogno, F. Romanelli, and A. Tenaglia, "A novel distributed architecture for unmanned aircraft systems based on robot operating system 2," *IET Cyber-Syst. Robot.*, vol. 5, no. 1, p. 12083, Mar. 2023.
[16] S.-C. Lin, Y. Zhang, C.-H. Hsu, M. Skach, M. E. Haque, L. Tang, and J. Mars, "The architectural implications of autonomous driving: Constraints and acceleration," in *Proc. 23rd Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Mar. 2018, pp. 751–766.
[17] D. Hong and C. Moon, "Autonomous driving system architecture with integrated ROS2 and adaptive AUTOSAR," *Electronics*, vol. 13, no. 7, p. 1303, Mar. 2024.
[18] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proc. ACM/IEEE 9th Int. Conf. Cyber-Phys. Syst. (ICCPS)*, Apr. 2018, pp. 287–296.
[19] The Autoware Foundation. *Autoware—The World's Leading Open-source Software Project for Autonomous Driving*. Accessed: Mar. 31, 2025. [Online]. Available: https://github.com/autowarefoundation/autoware
[20] V. Mayoral-Vilches, S. M. Neuman, B. Plancher, and V. J. Reddi, "RobotCore: An open architecture for hardware acceleration in ROS 2," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 9692–9699.
[21] C. Jung, A. Finazzi, H. Seong, D. Lee, S.-W. Lee, B. Kim, G. Kang, and D. H. Shim, "Autonomous system for head-to-head race: Design, implementation and analysis; team KAIST at the indy autonomous challenge," *IEEE Trans. Field Robot.*, early access, Nov. 18, 2024, doi: 10.1109/TFR.2024.3497922.
[22] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, Sep. 2004, pp. 2149–2154.
[23] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. Conf. robot Learn.*, Jan. 2017, pp. 1–16.
[24] G. Rong, B. H. Shin, H. Tabatabaee, Q. Lu, S. Lemke, M. Mozeiko, E. Boise, G. Uhm, M. Gerow, S. Mehta, E. Agafonov, T. H. Kim, E. Sterner, K. Ushiroda, M. Reyes, D. Zelenkovsky, and S. Kim, "LGSVL simulator: A high fidelity simulator for autonomous driving," in *Proc. IEEE 23rd Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2020, pp. 1–6.
[25] Tier.IV. (2023). *AWSIM: Open Source Simulator for Self-Driving Vehicles*. [Online]. Available: https://github.com/tier4/AWSIM
[26] G. Kaljavesi, T. Kerbl, T. Betz, K. Mitkovskii, and F. Diermeyer, "CARLA-autoware-bridge: Facilitating autonomous driving research with a unified framework for simulation and module development," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2024, pp. 224–229.
[27] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the performance of ROS2," in *Proc. Int. Conf. Embedded Softw. (EMSOFT)*, Oct. 2016, pp. 1–10.
[28] T. Wu, B. Wu, S. Wang, L. Liu, S. Liu, Y. Bao, and W. Shi, "Oops! It's too late. Your autonomous driving system needs a faster middleware," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7301–7308, Oct. 2021.
[29] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ROS 2," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6511–6518, Jul. 2022.
[30] P.-Y. Lajoie, C. Bédard, and G. Beltrame, "Analyze, debug, optimize: Real-time tracing for perception and mapping systems in ROS 2," 2022, *arXiv:2204.11778*.
[31] T. Kuboichi, A. Hasegawa, B. Peng, K. Miura, K. Funaoka, S. Kato, and T. Azumi, "CARET: Chain-aware ROS 2 evaluation tool," in *Proc. IEEE Int. Conf. Embedded Ubiquitous Comput. (EUC)*, Dec. 2022, pp. 123–130.
[32] Z. Li, A. Hasegawa, and T. Azumi, "Autoware_Perf: A tracing and performance analysis framework for ROS 2 applications," *J. Syst. Archit.*, vol. 123, Feb. 2022, Art. no. 102341. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762121002344
[33] T. Betz, M. Schmeller, A. Korb, and J. Betz, "Latency measurement for autonomous driving software using data flow extraction," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2023, pp. 1–8.
[34] T. Kronauer, J. Pohlmann, M. Matthe, T. Smejkal, and G. Fettweis, "Latency analysis of ROS2 multi-node systems," in *Proc. IEEE Int. Conf. Multisensor Fusion Integr. Intell. Syst. (MFI)*, Sep. 2021, pp. 1–7.
[35] Y.-P. Wang, W. Tan, X.-Q. Hu, D. Manocha, and S.-M. Hu, "TZC: Efficient inter-process communication for robotics middleware with partial serialization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 7805–7812.
[36] T. Betz, M. Schmeller, H. Teper, and J. Betz, "How fast is my software? Latency evaluation for a ROS 2 autonomous driving software," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2023, pp. 1–6.
[37] H. Teper, T. Betz, G. Von Der Brüggen, K.-H. Chen, J. Betz, and J.-J. Chen, "Timing-aware ROS 2 architecture and system optimization," in *Proc. IEEE 29th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2023, pp. 206–215.
[38] S. Macenski, A. Soragna, M. Carroll, and Z. Ge, "Impact of ROS 2 node composition in robotic systems," *IEEE Robot. Autom. Lett.*, vol. 8, no. 7, pp. 3996–4003, Jul. 2023.
[39] H. Wei, Z. Shao, Z. Huang, R. Chen, Y. Guan, J. Tan, and Z. Shao, "RT-ROS: A real-time ROS architecture on multi-core processors," *Future Gener. Comput. Syst.*, vol. 56, pp. 171–178, Jun. 2015.
[40] Y. Saito, F. Sato, T. Azumi, S. Kato, and N. Nishio, "ROSCH: Real-time scheduling framework for ROS," in *Proc. IEEE 24th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2018, pp. 52–58.
[41] Y. Tang, Z. Feng, N. Guan, X. Jiang, M. Lv, Q. Deng, and W. Yi, "Response time analysis and priority assignment of processing chains on ROS2 executors," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2020, pp. 231–243.
[42] A. A. Arafat, S. Vaidhun, K. M. Wilson, J. Sun, and Z. Guo, "Response time analysis for dynamic priority scheduling in ROS2," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 301–306.
[43] H. Lee, Y. Choi, T. Han, and K. Kim, "Probabilistically guaranteeing end-to-end latencies in autonomous vehicle computing systems," *IEEE Trans. Comput.*, vol. 71, no. 12, pp. 3361–3374, Dec. 2022.
[44] T. Han and K. Kim, "Minimizing probabilistic end-to-end latencies of autonomous driving systems," in *Proc. IEEE 29th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2023, pp. 27–39.
[45] M. Günzel, N. Ueter, K.-H. Chen, G. von der Brüggen, and J.-J. Chen, "Probabilistic reaction time analysis," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 5s, pp. 1–22, Sep. 2023, doi: 10.1145/3609390.
[46] H. Sobhani, H. Choi, and H. Kim, "Timing analysis and priority-driven enhancements of ROS 2 multi-threaded executors," in *Proc. IEEE 29th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2023, pp. 106–118.
[47] H. Teper, D. Kuhse, M. Günzel, G. V. D. Brüggen, F. Howar, and J.-J. Chen, "Thread carefully: Preventing starvation in the ROS 2 multithreaded executor," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 3588–3599, Nov. 2024.
[48] X. Luo, X. Jiang, N. Guan, H. Liang, S. Liu, and W. Yi, "Modeling and analysis of inter-process communication delay in ROS 2," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2023, pp. 198–209.
[49] G. Sciangula, D. Casini, A. Biondi, C. Scordino, and M. Di Natale, "Bounding the data-delivery latency of DDS messages in real-time applications," in *Proc. 35th Euromicro Conf. Real-Time Syst. (ECRTS)* (Leibniz International Proceedings in Informatics (LIPIcs)), vol. 262. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 2023, pp. 9:1–9:26. [Online]. Available: https://doi.org/10.4230/LIPIcs.ECRTS.2023.9
[50] SOAFEE. (2022). *SOAFEE: Scalable Open Architecture for Embedded Edge*. [Online]. Available: https://www.soafee.io
[51] S. Giallorenzo, J. Mauro, M. G. Poulsen, and F. Siroky, "Virtualization costs: Benchmarking containers and virtual machines against bare-metal," *Social Netw. Comput. Sci.*, vol. 2, no. 5, p. 404, Aug. 2021.
[52] N. Kukulicic, D. Samardzic, A. Bucaioni, and S. Mubeen, "Automotive service-oriented architectures: A systematic mapping study," in *Proc. 48th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Aug. 2022, pp. 459–466.
[53] M. G. Xavier, M. V. Neves, F. D. Rossi, T. Ferreto, T. Lange, and C. A. F. De Rose, "Performance evaluation of container-based virtualization for high performance computing environments," in *Proc. 21st Euromicro Int. Conf. Parallel, Distrib., Netw.-Based Process.*, Feb. 2013, pp. 233–240.

[54] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, p. 2, Mar. 2014.
[55] J. Lotz, A. Vogelsang, O. Benderius, and C. Berger, "Microservice architectures for advanced driver assistance systems: A case-study," in *Proc. IEEE Int. Conf. Softw. Archit. Companion (ICSA-C)*, Mar. 2019, pp. 45–52.
[56] D. Neri, J. Soldani, O. Zimmermann, and A. Brogi, "Design principles, architectural smells and refactorings for microservices: A multivocal review," *SICS Softw.-Intensive Cyber-Phys. Syst.*, vol. 35, nos. 1–2, pp. 3–15, Aug. 2020.
[57] L. Wen, M. Rickert, F. Pan, J. Lin, and A. Knoll, "Bare-metal vs. hypervisors and containers: Performance evaluation of virtualization technologies for software-defined vehicles," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2023, pp. 1–8.
[58] T. Betz, L. Wen, F. Pan, G. Kaljavesi, A. Zuepke, A. Bastoni, M. Caccamo, A. Knoll, and J. Betz, "A containerized microservice architecture for a ROS 2 autonomous driving software: An end-to-end latency evaluation," in *Proc. IEEE 30th Int. Conf. Embedded Real-Time Comput. Syst. Appl. (RTCSA)*, Aug. 2024, pp. 57–66.
[59] A. Mehran and D. Ulus, "Runtime verification containers for publish/subscribe networks," 2024, *arXiv:2408.06380*.
[60] Y. G. Choi, S. W. Lee, and J. W. Jeon, "Analysis of ROS2-based automotive architecture using containers for SDV," in *Proc. IEEE 33rd Int. Symp. Ind. Electron. (ISIE)*, Jun. 2024, pp. 1–5.
[61] F. Pan, M. Rickert, T. Betz, L. Wen, J. Lin, N. Petrovic, M. Lienkamp, and A. Knoll, "Towards software-defined vehicles: From model-based engineering to virtualization-based deployment," *IEEE Access*, vol. 13, pp. 192127–192145, 2024.
[62] L. Wen, M. Rickert, F. Pan, J. Lin, Y. Zhang, T. Betz, and A. Knoll, "Virtualization & microservice architecture for software-defined vehicles: An evaluation and exploration," 2024, *arXiv:2412.09995*.
[63] I. Ahmad, M. G. AlFailakawi, A. AlMutawa, and L. Alsalman, "Container scheduling techniques: A survey and assessment," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 7, pp. 3934–3947, Jul. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1319157821000562
[64] E. A. Brewer, "Kubernetes and the path to cloud native," in *Proc. 6th ACM Symp. Cloud Comput.*, Aug. 2015, p. 167.
[65] Rancher Labs. *K3s—Leightweight Kubernetes*. Accessed: Mar. 31, 2025. [Online]. Available: https://github.com/k3s-io/k3s/
[66] N. Ueter, M. Günzel, and J.-J. Chen, "Response-time analysis and optimization for probabilistic conditional parallel DAG tasks," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2021, pp. 380–392.
[67] S. Shi, X. Wang, and H. Li, "PointRCNN: 3D object proposal generation and detection from point cloud," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jan. 2018, pp. 770–779.
[68] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: In defense of point-to-point ICP—Simple, accurate, and robust registration if done the right way," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 1029–1036, Feb. 2023.
[69] P. Karle, F. Fent, S. Huch, F. Sauerbeck, and M. Lienkamp, "Multi-modal sensor fusion and object tracking for autonomous racing," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 7, pp. 3871–3883, Jul. 2023.
[70] L. Ögretmen, M. Rowold, A. Langmann, and B. Lohmann, "Sampling-based motion planning with online racing line generation for autonomous driving on three-dimensional race tracks," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2024, pp. 811–818.
[71] A. Wischnewski, T. Herrmann, F. Werner, and B. Lohmann, "A tube-MPC approach to autonomous multi-vehicle racing on high-speed ovals," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 1, pp. 368–378, Jan. 2023.
[72] S. Goblirsch, M. Weinmann, and J. Betz, "Three-dimensional vehicle dynamics state estimation for high-speed race cars under varying signal quality," 2024, *arXiv:2408.14885*.
[73] Eclipse Foundation. (2022). *Eclipse Cyclone DDS*. [Online]. Available: https://cyclonedds.io
[74] Autonomous Vehicle Systems Lab. (2023). *Ros2_Latency_Analysis*. Accessed: Jun. 5, 2025. [Online]. Available: https://github.com/TUM-AVS/ros2_latency_analysis
[75] Gurobi Optimization, LLC. (2024). *Gurobi Optimizer Reference Manual*. [Online]. Available: https://www.gurobi.com



TOBIAS BETZ received the B.Eng. degree from the University of Applied Science Regensburg, in 2018, and the M.Sc. degree from the Technical University of Munich (TUM), in 2020, where he is currently pursuing the Ph.D. degree with the Institute of Automotive Technology. His research interests include ROS 2, latency analysis, system optimization, and software deployment on real-world applications in autonomous driving.



HARUN TEPER (Member, IEEE) received the B.Sc. and M.Sc. degrees from TU Dortmund University (TUD), in 2019 and 2021, respectively, where he is currently pursuing the Ph.D. degree with the Design Automation for Embedded Systems Group. His research interests include real-time systems, ROS 2, end-to-end latency analysis, latency optimization, and autonomous driving.



DOMINIC EBNER received the B.Sc. and M.Sc. degrees from the Technical University of Munich (TUM), in 2020 and 2023, respectively, where he is currently pursuing the Ph.D. degree with the Institute of Automotive Technology. His research interests include simulation, object detection for autonomous vehicles, and real-world applications of both for autonomous driving.



MAXIMILIAN LEITENSTERN received the B.Sc. degree from the Technical University of Munich (TUM), in 2021, and the M.Sc. degree, in 2023, where he is currently pursuing the Ph.D. degree with the Institute of Automotive Technology. His research interests include LiDAR- and RaDAR-based localization, mapping, and sensor fusion for real-world applications in autonomous driving.



SIMON SAGMEISTER (Graduate Student Member, IEEE) received the B.Sc. degree from the Technical University of Munich (TUM), in 2019, and the M.Sc. degree, in 2021, where he is currently pursuing the Ph.D. degree with the Institute of Automotive Technology. His research interests include vehicle simulation as well as development and evaluation of control algorithms in autonomous driving applications.



MARCEL WEINMANN received the B.Eng. degree from Baden-Wuerttemberg Cooperative State University (DHBW), in 2020, and the M.Sc. degree from the Technical University of Munich (TUM), in 2020, where he is currently pursuing the Ph.D. degree with the Institute of Automotive Technology. His research interests include enhancing scene understanding and improving vehicle localization for real-world autonomous driving applications.



JIAN-JIA CHEN (Senior Member, IEEE) received the B.S. degree from the Department of Chemistry, National Taiwan University, in 2001, and the Ph.D. degree from the Department of Computer Science and Information Engineering, National Taiwan University, Taiwan, in 2006. He was a Junior Professor with the Department of Informatics, Karlsruhe Institute of Technology (KIT), Germany, from May 2010 to March 2014. He is currently a Professor with the Department of Informatics, TU Dortmund University, Germany. His research interests include real-time systems, embedded systems, energy-efficient scheduling, power-aware designs, temperature-aware scheduling, and distributed computing. He received the European Research Council (ERC) Consolidator Award, in 2019. He has received more than ten best paper awards and outstanding paper awards and has involved in technical committees in many international conferences.



MARKUS LIENKAMP (Member, IEEE) received the Ph.D. degree from TU Darmstadt University, in 1995. He was appointed as the Head of the Electronics and Vehicle Research Department, Volkswagen Group's Research Division. He has headed the Institute of Automotive Technology, Technical University of Munich (TUM), since November 2009. He is currently a Professor at the Institute of Automotive Technology, TUM. He carries out research in the area of autonomous vehicles with the objective of creating an open-source software platform. He studying mechanical engineering at TU Darmstadt University and Cornell University. He worked at Volkswagen as part of an international trainee program and took part in a joint venture between Ford and Volkswagen in Portugal. Returning to Germany, he led the brake testing department of the VW commercial vehicle development section in Wolfsburg.